

"The API contract is the handshake -- the UI is only as reliable as the promise it trusts."

MODULE 35

Frontend to API Integration

Connect the Angular client to the Spring Boot API with typed, observable-based HTTP calls.





MODULE 35 OVERVIEW

Wire the Angular client to the Spring Boot REST API using HttpClient, typed models, and interceptors.

Northstar CRM's Angular UI must call the Spring Boot customer API, show real loading/empty/error states, and never leak an unhandled network failure to the screen.

DURATION & LAB



3 Hours Theory

HttpClient, typed observables, interceptors, environment config, and CORS across the Angular-to-Spring Boot boundary.



1 Hour Lab

Build an ApiService with typed CRUD methods, an auth interceptor, and loading/empty/error UI states.



Scenario

The CRM customer list screen must call GET /api/customers with pagination, sorting, and filtering.

MODULE ARC



Call the API

HttpClient methods, typed responses, and RxJS observables replace manual fetch/XHR plumbing.



Handle Every State

Loading, empty, and error states are first-class UI states, not afterthoughts.



Secure and Configure

Interceptors inject auth tokens; environment files and CORS keep dev and prod calls working.

KEY TAKEAWAY Total time: 3 hours theory + 1 hour lab. A typed HttpClient service with interceptors and real loading/empty/error states is what separates a demo screen from a production-ready one.

WHY FRONTEND-BACKEND INTEGRATION MATTERS

A correct Angular component is worthless if it cannot reliably talk to the API.



Two Apps, One Contract

Angular and Spring Boot ship separately but must agree on every URL and field name -- integration is where mismatches surface.



The Weakest Link

A perfect component with a broken HTTP call is still a broken feature -- users only see the screen, not the layers behind it.



Where Real Bugs Live

CORS errors, stale DTOs, and missed loading states all happen at this exact seam, not inside either app alone.

KEY TAKEAWAY This module is about the seam between two working apps -- every topic here explains a way that seam breaks in production.



Why Frontend-Backend Integration Matters

Great applications are built when the frontend and backend work together **seamlessly**.

Integration turns isolated layers into a **complete, reliable, and valuable** solution for users and the business.



Seamless User Experience

Users interact with the UI while data, rules, and processes are handled securely on the backend.



Data Consistency & Accuracy

Real-time communication ensures users always see up-to-date and accurate information.



Business Logic Centralization

Complex rules, validations, and workflows live in the backend, keeping the frontend lightweight and focused.



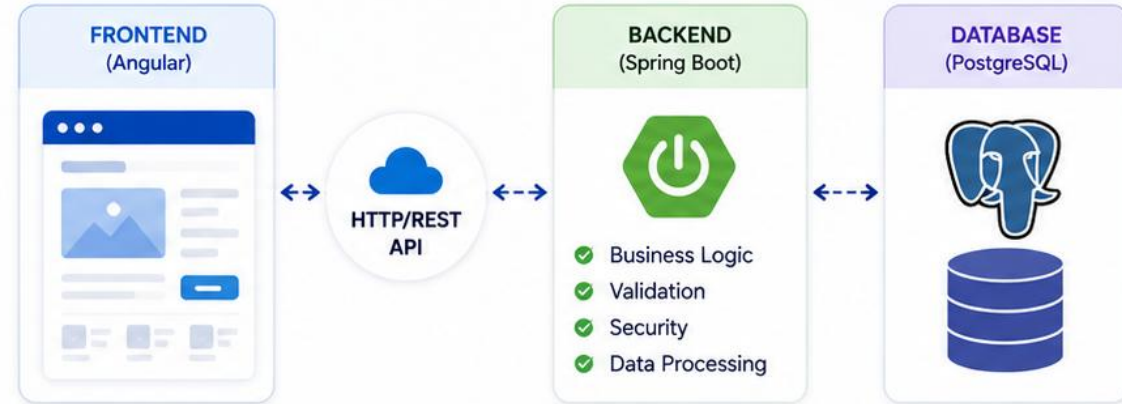
Security & Compliance

Sensitive data stays protected on the server with proper authentication, authorization, and audit controls.



Scalability & Maintainability

Well-defined APIs and clear separation of concerns enable independent scaling and easier long-term maintenance.



End-to-End Value



User Action

User interacts with the UI (clicks, submits, views data).



Request Sent

Frontend sends an HTTP request to the backend API.



Backend Processes

Backend validates, applies business logic, and accesses the database.



Data Returned

Backend returns structured data or a response.



UI Updated

Frontend updates the UI to reflect the latest state.



Integration bridges the gap between **user needs** and **business capabilities**.



Module 35: Frontend to API Integration



Goal: Build reliable, secure, and scalable full-stack applications.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to:

1. Configure Angular's HttpClient with provideHttpClient() and call a Spring Boot REST API over GET, POST, PUT, PATCH, and DELETE.
2. Build a typed ApiService using TypeScript interfaces that mirror the backend's DTO shapes field-for-field.
3. Subscribe to HTTP observables correctly, and prefer the AsyncPipe to avoid manual subscription leaks.
4. Model loading, empty, and error states explicitly in every component that calls the API.
5. Write HTTP interceptors that inject an Authorization header and centralize error handling.
6. Configure environment-based API URLs so dev, test, and production builds point at the right backend.
7. Diagnose and fix CORS failures on both the Angular and Spring Boot sides of a blocked request.
8. Wire pagination, sorting, and filtering parameters through the service layer to a Spring Data-backed endpoint.

KEY TAKEAWAY These eight objectives take you from an Angular component with no backend connection to one that calls a real, secured, paginated Spring Boot API and handles every outcome of that call gracefully.

Learning Objectives

By the end of this module, you will be able to integrate an **Angular** frontend with **Spring Boot REST APIs** and display data from **PostgreSQL** reliably, securely, and efficiently.



Goal

Build seamless, secure, and maintainable communication between the frontend and backend in a full-stack application.

1



Understand the Integration Flow

Explain how Angular communicates with Spring Boot REST APIs and how data is persisted in PostgreSQL.

2



Use Angular HttpClient Effectively

Perform CRUD operations using HttpClient with observables and handle API responses.

3



Design a Service Layer

Create reusable, typed API services for clean and maintainable communication.

4



Handle Data and UI States

Manage loading, success, empty, and error states for a great user experience.

5



Implement Security Best Practices

Use HTTP interceptors, tokens, and CORS to secure API communication.

6



Integrate Advanced API Features

Implement pagination, sorting, and filtering in the frontend.

7



Debug and Optimize Integration

Use browser tools and best practices to diagnose issues and improve performance.

ANGULAR FRONTEND



HTTP/REST
JSON

SPRING BOOT API



- ✓ Business Logic
- ✓ Validation
- ✓ Security
- ✓ Data Processing

JPA/Hibernate
SQL

POSTGRESQL DATABASE



Real-Time Data

Always show the latest information to users.



Reliability

Ensure consistency and accuracy across layers.



Efficiency

Reduce duplication and improve performance.



Better UX

Deliver a fast, secure, and responsive application.



Great integrations create great applications. Let's build them the right way!



FULL-STACK ARCHITECTURE OVERVIEW

Three tiers, one request path: Angular renders, Spring Boot decides, PostgreSQL persists.



Angular Client

Standalone components call a typed ApiService; the browser owns state, routing, and rendering -- never direct database access.



Spring Boot API

Controllers, services, and repositories enforce validation, business rules, and security before anything touches the database.



PostgreSQL

The system of record. Every write goes through a transactional service method, never straight from a controller.

KEY TAKEAWAY Angular never talks to PostgreSQL directly -- every request crosses the network as HTTP/JSON and is validated by Spring Boot before it reaches the database.



Full-Stack Architecture Overview

A modern full-stack application is built by integrating the frontend, backend, and database into a cohesive system that delivers value to users.



End-to-End Solution

All layers work together to provide a seamless user experience.



Separation of Concerns

UI/UX in the frontend, business logic in the backend, data persistence in the database.



Scalable & Maintainable

Independent layers can be developed, deployed, scaled, and maintained separately.



Secure Communication

APIs act as a secure contract between the frontend and backend layers.



ANGULAR -> SPRING BOOT -> POSTGRESQL FLOW

Trace one customer-creation request across all three tiers.

1

Angular Sends

A form component calls `apiService.createCustomer(dto)`, which issues `http.post<Customer>(url, dto)` and returns an `Observable`.

2

Spring Boot Processes

`@Valid` triggers Bean Validation, the service opens a transaction, and the repository issues an `INSERT` through Spring Data JPA.

3

PostgreSQL Persists

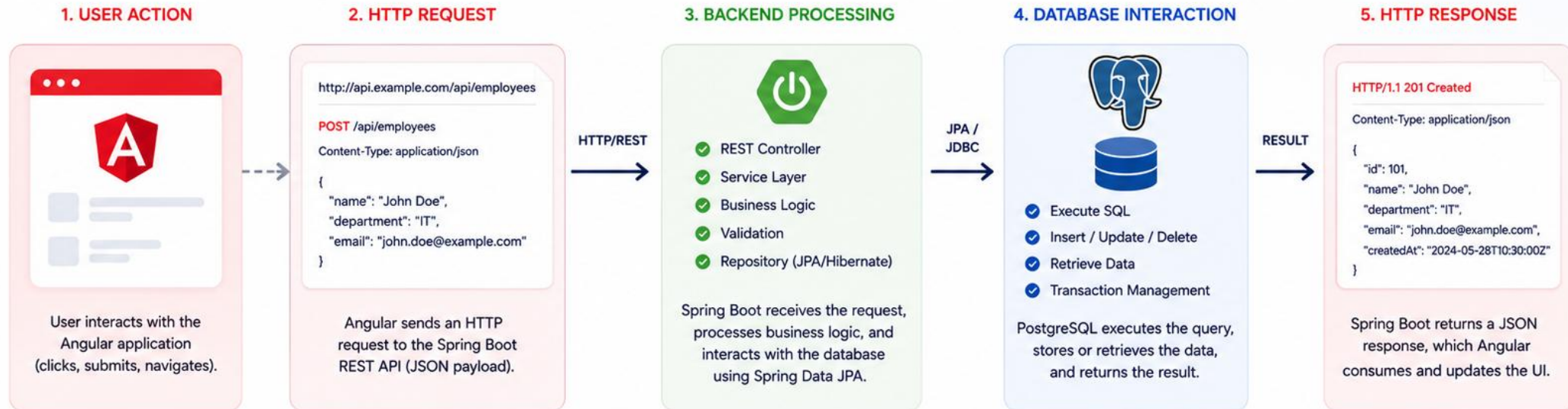
The row commits, an ID returns up the stack, and Spring Boot responds `201 Created` with a `Location` header and the DTO.

KEY TAKEAWAY One logical action -- 'create a customer' -- touches three independently deployed systems; every one of them must agree on the JSON shape for the request to succeed.



Angular → Spring Boot → PostgreSQL Flow

A request flows from the Angular frontend to Spring Boot REST APIs, which process the request and persist or retrieve data from PostgreSQL.



Flow Summary



User action in Angular UI



HTTP request sent to API



Spring Boot processes the request



Data is read from or written to PostgreSQL



Response returned to Angular UI

Key Benefits



Separation of concerns



Scalable architecture



Secure communication



Maintainable and testable



Real-time data consistency



Module 35: Frontend to API Integration

Angular Frontend



Spring Boot REST API



PostgreSQL Database

6

REST COMMUNICATION RECAP

A quick refresher on the REST principles this module builds directly on.

PRINCIPLE	WHAT IT MEANS FOR THIS API
Resources, not actions	/api/customers/{id} names a thing, not a verb -- the HTTP method supplies the action.
Stateless requests	Every request carries everything Spring Boot needs (auth token, body, params) -- no server-side session.
Uniform status codes	200/201/204 for success, 400/401/403/404/409 for client error, 500 for server error.
JSON as the media type	Content-Type: application/json on both request and response bodies, enforced by Spring Boot.
Idempotency by verb	GET/PUT/DELETE are safe to retry; POST is not -- this shapes Angular's retry strategy later in the module.

KEY TAKEAWAY REST is the shared vocabulary between Angular and Spring Boot -- every HttpClient call in this module assumes these five rules hold on the server side.



REST Communication Recap

REST (Representational State Transfer) enables systems to communicate over HTTP using a **stateless**, **resource-based**, and **uniform interface**.



REST Principles



Client-Server

Separation of concerns between UI and data.



Stateless

Each request contains all information needed.



Cacheable

Responses can be cached to improve performance.



Uniform Interface

Consistent way to interact with resources.



Layered System

Intermediary layers can exist without exposing details.



Resource-Based

Everything is a resource identified by a URI.



REST Communication Flow



Client (Angular)
User interacts with the application.

HTTP Request
GET /employees
Accept: application/json



Server (Spring Boot API)
Processes the request and applies business logic.

HTTP Response
200 OK
Content-Type: application/json



Database (PostgreSQL)
Reads or writes data and returns result.

JSON Response



HTTP Methods

GET

Retrieve a resource

POST

Create a new resource

PUT

Replace an existing resource

PATCH

Partially update a resource

DELETE

Remove a resource



Key Characteristics

- ✓ Uses standard HTTP methods and status codes
- ✓ Communicates using JSON (most common)
- ✓ Stateless: no client context on the server
- ✓ Supports caching, scalability, and performance
- ✓ Works over any HTTP-compatible protocol



Example Request

```
GET /api/employees/101 HTTP/1.1
Host: api.example.com
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```



Every request is independent and contains all the information needed.



Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "id": 101,
  "name": "John Doe",
  "department": "IT",
  "email": "john.doe@example.com"
}
```



Module 35: Frontend to API Integration

Angular (Frontend)



Spring Boot REST API



PostgreSQL Database

7

JSON REQUEST AND RESPONSE FLOW

Every HTTP call in this module is a JSON object going out and a JSON object coming back.



Request Body

Angular serializes a TypeScript object to JSON automatically once it's passed as an HttpClient body argument.



Response Body

Spring Boot serializes a DTO to JSON via Jackson; HttpClient parses it back into a typed object on the Angular side.



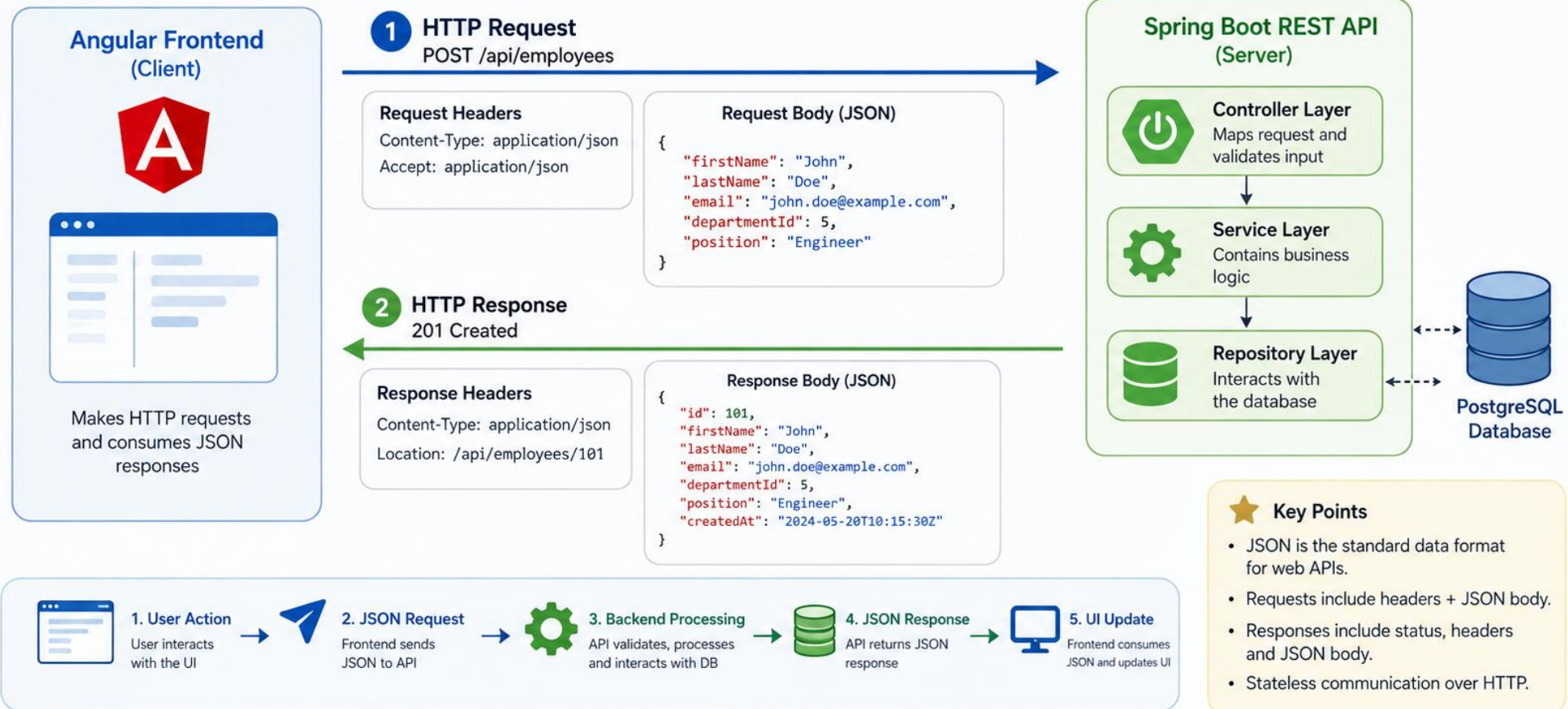
Content-Type Header

application/json on both sides is what makes automatic (de)serialization work by default.

KEY TAKEAWAY Nobody hand-writes `JSON.stringify()` or `JSON.parse()` in this workflow -- HttpClient and Jackson do it automatically as long as the Content-Type header and the object shapes agree.

JSON Request and Response Flow

Frontend sends a JSON request to the REST API.
Backend processes it and returns a JSON response.



ANGULAR HTTPCLIENT OVERVIEW

HttpClient is Angular's built-in, typed, observable-based HTTP client -- the only supported way to call a REST API.



From @angular/common/http

HttpClient replaces raw fetch()/XMLHttpRequest with typed, testable, interceptor-aware requests built on RxJS.



Returns Observables

Every method (get, post, put, patch, delete) returns a cold Observable -- nothing happens until something subscribes.



Testable by Design

HttpClientTestingModule lets tests assert exactly which requests were made without hitting a real network or backend.

KEY TAKEAWAY HttpClient is not optional infrastructure -- it is the single entry point every service in this module uses to reach the Spring Boot API.



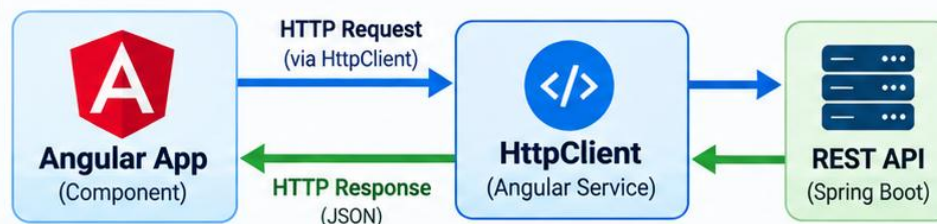
Angular HttpClient Overview

HttpClient is Angular's built-in service for making HTTP requests to a REST API and handling the responses.

It provides a clean, observable-based API for working with HTTP data in a reactive way.

Key Features

-  **HTTP Methods**
Supports GET, POST, PUT, PATCH, DELETE
-  **Observable-Based**
Uses RxJS Observables for async data
-  **Configurable**
Set headers, params, and options
-  **Interceptors**
Handle auth, logging, and global errors
-  **Typed Responses**
Works with TypeScript interfaces
-  **Easy Integration**
Works seamlessly with Angular services



Example: GET Request

TypeScript

```
import { HttpClient, HttpParams } from '@angular/common/http';
import { Observable } from 'rxjs';

export class EmployeeService {
  constructor(private http: HttpClient) {}

  getEmployees(): Observable<any[]> {
    return this.http.get<any[]>('/api/employees');
  }
}
```

Common Operations

- GET** Retrieve data
`http.get<T>(url)`
- POST** Create data
`http.post<T>(url, body)`
- PUT** Replace data
`http.put<T>(url, body)`
- PATCH** Partial update
`http.patch<T>(url, body)`
- DELETE** Remove data
`http.delete<T>(url)`

Why It Matters



Simplifies
HTTP communication



Enables
reactive programming



Supports
authentication & error handling



Works
with Angular services

CONFIGURING HTTPCLIENT

HttpClient must be provided once at bootstrap before any service can inject it.

CONFIGURATION STEP	WHAT IT DOES
provideHttpClient()	Registers HttpClient in app.config.ts for a standalone Angular application.
withInterceptors([...])	Attaches functional interceptors (auth, error handling) to every outgoing request.
withFetch() (optional)	Switches HttpClient's transport to the native fetch API instead of XMLHttpRequest.
Injecting HttpClient	inject(HttpClient) or constructor injection inside any @Injectable service.
HttpClientTestingModule	Provided only in spec files to intercept and assert requests in unit tests.

KEY TAKEAWAY `provideHttpClient()` belongs in `app.config.ts` exactly once -- every `ApiService` in the app then injects the same configured `HttpClient` instance, interceptors included.

Configuring HttpClient



HttpClient must be configured and provided to make HTTP requests in Angular applications.



1. Import HttpClientModule

HttpClientModule provides the HttpClient service and related providers.

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

import { HttpClientModule } from '@angular/common/http';
import { AppComponent } from './app.component';

@NgModule({
  declarations: [AppComponent],
  imports: [
    BrowserModule,
    HttpClientModule // <-- Import here
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```



2. Provide at Root (Optional in Standalone)

In Angular 15+ with standalone APIs, you can provide HttpClient in the application configuration.

```
// main.ts (Angular 15+ with standalone bootstrap)
import { bootstrapApplication } from '@angular/platform-browser';
import { provideHttpClient } from '@angular/common/http';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, {
  providers: [
    provideHttpClient() // provides HttpClient at root level
  ]
});
```



3. Inject HttpClient in a Service

Inject HttpClient to use it for making HTTP requests.

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class EmployeeService {
  constructor(private http: HttpClient) {}

  getEmployees(): Observable<any[]> {
    return this.http.get<any[]>('/api/employees');
  }
}
```



HttpClient Configuration Options

You can configure HttpClient with custom options such as headers, timeouts, and base URL.

Headers

```
const headers = new HttpHeaders({
  'Content-Type': 'application/json',
  'Authorization': 'Bearer token'
});
this.http.get('/api/data', { headers });
```

Timeout

```
this.http.get('/api/data', {
  timeout: 10000 // 10 seconds
}).subscribe();
```



Base URL via Environment

```
// environment.ts
export const environment = {
  apiUrl: 'https://api.example.com'
};
```



4. Recommended Best Practices

- ✓ Import HttpClientModule once in the root module.
- ✓ Use provideHttpClient() with standalone applications.
- ✓ Inject HttpClient in services, not in components.
- ✓ Centralize API base URL in environment configuration.
- ✓ Use HTTP Interceptors for cross-cutting concerns (auth, logging, error handling).



Key Takeaway

HttpClient must be imported or provided, injected, and optionally configured with headers, timeouts, and base URL to communicate with REST APIs efficiently.



Import
HttpClientModule



Provide
HttpClient



Inject
HttpClient



Make HTTP
Requests



REST API
(Server)

ANGULAR SERVICE LAYER

Components never call HttpClient directly -- a service layer owns every API call.



Single Responsibility

One ApiService (or one per resource) owns all HTTP calls for that resource -- components stay focused on presentation.



Reusable Across Components

The customer list, detail, and edit screens all inject the same CustomerApiService instead of duplicating HTTP logic.



Mockable in Tests

Component tests can provide a fake CustomerApiService, isolating component logic from real HTTP calls entirely.

KEY TAKEAWAY Every HttpClient call in this course's Angular app lives inside an @Injectable service -- never directly inside a component's TypeScript class.

Angular Service Layer



Services in Angular encapsulate business logic and handle communication with REST APIs using HttpClient. Components consume services to get data and perform actions.

★ Why Use a Service Layer?

- ✓ Separates concerns – components focus on UI, services handle data and business logic
- ✓ Reusable – same service can be used by multiple components
- ✓ Testable – easy to mock services in unit tests
- ✓ Maintainable – centralize API calls and common logic
- ✓ Consistent – uniform error handling and response mapping

Typical Folder Structure



Role of the Service Layer



Example Service: employee.service.ts

```
import { Injectable } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Employee, EmployeeCreateDto, EmployeeUpdateDto } from '../models/employee.model';

@Injectable({
  providedIn: 'root'
})
export class EmployeeService {
  private readonly baseUrl = `${environment.apiUrl}/api/employees`;

  constructor(private http: HttpClient) {}

  getEmployees(): Observable<Employee[]> {
    return this.http.get<Employee[]>(this.baseUrl);
  }

  getEmployee(id: number): Observable<Employee> {
    return this.http.get<Employee>(`${this.baseUrl}/${id}`);
  }

  // ... createEmployee, updateEmployee, deleteEmployee, etc.
}
```

Service Responsibilities

- Build API Requests**
Construct URLs, set headers, query params, and request body.
- Handle Responses**
Map and transform API responses into models.
- Handle Errors**
Centralize error handling and return user-friendly messages.
- Encapsulate Logic**
Add business rules, caching, retry logic, or utility methods.

Example Usage in a Component

```
export class EmployeeListComponent implements OnInit {
  employees: Employee[] = [];
  loading = true;
  error?: string;

  constructor(private employeeService: EmployeeService) {}

  ngOnInit(): void {
    this.loadEmployees();
  }

  loadEmployees(): void {
    this.loading = true;
    this.employeeService.getEmployees().subscribe({
      next: (data) => {
        this.employees = data;
        this.loading = false;
      },
      error: (err) => {
        this.error = 'Failed to load employees';
        this.loading = false;
      }
    });
  }
}
```

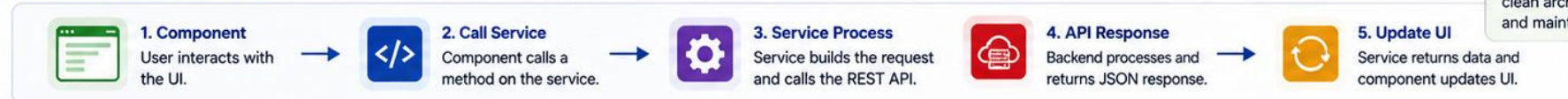
Best Practices

- ✓ Provide services in root unless scoped services are needed.
- ✓ Keep services focused on a single responsibility.
- ✓ Return Observables; let components subscribe.
- ✓ Use typed models and interfaces.
- ✓ Handle errors in the service or via interceptors.
- ✓ Keep API endpoints in environment configuration.

💡 Key Takeaway

The service layer is the bridge between UI components and backend APIs. It promotes clean architecture, reusability, and maintainability.

Flow: Component to REST API



CREATING AN API SERVICE

A minimal, typed CustomerApiService -- the pattern every resource service in this course follows.

ELEMENT	PURPOSE
<code>@Injectable({ providedIn: 'root' })</code>	Registers the service as a singleton available app-wide without a module declaration.
<code>private http = inject(HttpClient);</code>	Injects HttpClient using Angular's functional inject() API.
<code>private baseUrl = `\${environment.apiUrl}/customers`;</code>	Builds the resource URL from the active environment config.
<code>getCustomers(): Observable<Customer[]></code>	Typed GET method returning an Observable of the resource's array shape.
<code>createCustomer(dto: CreateCustomerRequest)</code>	Typed POST method that accepts a request DTO and returns the created resource.

KEY TAKEAWAY Every method on an API service declares its return type as `Observable<T>` for a specific `T` -- this is what makes 'typed responses' possible everywhere else in the module.

TRY IT YOURSELF — EXERCISE 1: PLAN THE API SERVICE

Reinforces: the full-stack flow, HttpClient configuration, and the Angular service layer you just walked, planned for CustomerApiService.

STEPS (notes/lab35-api-service.md)

Step	What To Do
1 — Provide HttpClient	Say where HttpClient is provided in a standalone Angular application.
2 — Service Shape	Name the service, how it is injectable, and what it injects.
3 — Method List	List the methods it exposes and the HTTP call each makes.
4 — Boundary	State what the service must not do: no templates, no component state.

Service sketch

```
app.config.ts: providers: [provideHttpClient()]
@Injectable({providedIn:'root'}) class CustomerApiService
  list() get() create() update() remove()  -- returns Observables, nothing more
```

TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-api-service-plan.md (workspace: examples/module-35-exercises).

Creating an API Service



An API service in Angular encapsulates HTTP calls to backend REST endpoints and exposes methods that components can use.

1 Generate the Service

```
ng generate service core/services/employee
```

```
src/app
├── core
│   └── services
│       └── employee.service.ts
```

2 Define the Model (Example)

```
// employee.model.ts
export interface Employee {
  id: number;
  firstName: string;
  lastName: string;
  email: string;
  department: string;
  position: string;
}
```

3 Create the Service

```
// employee.service.ts
import { Injectable } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Employee } from '../models/employee.model';

@Injectable({
  providedIn: 'root'
})
export class EmployeeService {
  private baseUrl = environment.apiUrl + '/api/employees';

  constructor(private http: HttpClient) {}

  // Methods will be added in the next slides
}
```

What We Did

- ✓ Generated a service using Angular CLI
- ✓ Defined a TypeScript interface for Employee
- ✓ Created the service class
- ✓ Injected HttpClient to make HTTP requests
- ✓ Defined the base URL for the API endpoints

Best Practices

- ✓ Keep services focused on a single feature.
- ✓ Use typed models and interfaces.
- ✓ Centralize API base URL using environment variables.
- ✓ Return Observables for async handling.
- ✓ Handle errors in the service or via interceptors.

4 Use Environment for Base URL

```
// environment.ts
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8880'
};

// environment.prod.ts
export const environment = {
  production: true,
  apiUrl: 'https://api.myapp.com'
};
```

5 Provide HttpClient (If not already)

```
// app.config.ts (Angular 15+)
import { provideHttpClient } from '@angular/common/http';
import { ApplicationConfig } from '@angular/core';
import { appRoutes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [provideHttpClient()],
};
```

6 Inject and Use in a Component

```
constructor(private employeeService: EmployeeService) {}

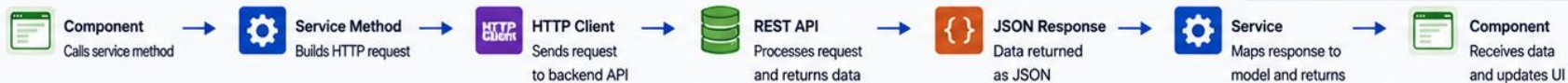
ngOnInit(): void {
  this.employeeService.getEmployees()
    .subscribe(employees => this.employees = employees);
}

employees: Employee[] = [];
```

Key Takeaway

An API service acts as a bridge between the Angular application and the backend. It promotes clean, reusable, and maintainable code by separating HTTP logic from components.

How It Works



GET / POST / PUT / PATCH / DELETE REQUESTS

Five HttpClient methods, five REST verbs, five distinct outcomes on the Spring Boot side.



GET

Read-only, safe to retry,
no request body.
Typically returns 200.



POST

Creates a resource. Not
idempotent. Typically
returns 201 + Location.



PUT

Full replace of a
resource. Idempotent.
Returns 200 or 204.



PATCH

Partial update, only
changed fields sent.
Returns 200.



DELETE

Removes a resource.
Idempotent. Returns
204, no body.

KEY TAKEAWAY PUT and DELETE are idempotent (safe to retry); POST is not -- this single distinction drives every retry and error-recovery decision later in the module.

GET Requests

Retrieving Resources from the API



GET is used to retrieve data from the server. It is **safe** and **idempotent**.



What is a GET Request?

GET requests data from the server without modifying any resource. Ideal for fetching lists, details, and filtered data.

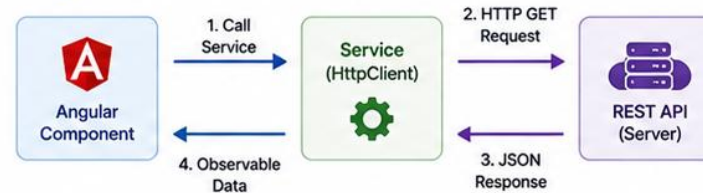


Key Characteristics

- ✓ **Safe** – does not change server state
- ✓ **Idempotent** – same request returns the same result
- ✓ Can be cached
- ✓ Data is sent in URL (query params)



GET Request Flow



Common Use Cases

- Get all employees
- Get employee by ID
- Search or filter employees
- Get paginated data
- Populate dropdowns

Service Method Example

```
// employee.service.ts
getEmployees(): Observable<Employee[]> {
  return this.http.get<Employee[]>(`${this.baseUrl}/employees`);
}

getEmployeeById(id: number): Observable<Employee> {
  return this.http.get<Employee>(`${this.baseUrl}/employees/${id}`);
}

searchEmployees(dept: string, page: number = 0, size: number = 10):
Observable<Employee[]> {
  const params = new HttpParams()
    .set('department', dept)
    .set('page', page.toString())
    .set('size', size.toString());
  return this.http.get<Employee[]>(`${this.baseUrl}/employees`, { params });
}
```

Example Component Usage

```
export class EmployeeListComponent implements OnInit {
  employees: Employee[] = [];
  loading = true;
  error: string = '';

  constructor(private employeeService: EmployeeService) {}

  ngOnInit(): void {
    this.loadEmployees();
  }

  loadEmployees(): void {
    this.loading = true;
    this.employeeService.getEmployees().subscribe({
      next: (data) => {
        this.employees = data;
        this.loading = false;
      },
      error: (err) => {
        this.error = 'Failed to load employees';
        this.loading = false;
      }
    });
  }
}
```

HTTP Request Example

GET All Employees

GET https://api.example.com/api/employees

With Query Parameters

GET https://api.example.com/api/employees?department=Engineering&page=0&size=10

With Path Variable

GET https://api.example.com/api/employees/101

Headers (Optional)

Accept: application/json

Sample JSON Response

```
{
  {
    "id": 1,
    "firstName": "John",
    "lastName": "Doe",
    "email": "john.doe@example.com",
    "department": "Engineering"
  },
  {
    "id": 2,
    "firstName": "Jane",
    "lastName": "Smith",
    "email": "jane.smith@example.com",
    "department": "HR"
  }
}
```



Best Practices

- ✓ Use typed interfaces for responses
- ✓ Use query parameters for filtering, pagination, and sorting
- ✓ Handle loading, success, and error states
- ✓ Use the AsyncPipe in templates when possible
- ✓ Keep API base URL in environment configuration



Tips

- Use HttpParams for building query strings
- Avoid hardcoding URLs
- Return Observables and subscribe in the component
- Use caching headers on the server for GET endpoints



Error Handling Example

```
this.employeeService.getEmployees().subscribe({
  next: data => this.employees = data,
  error: err => this.error = err.message || 'An error occurred'
});
```



POST Requests

Creating Resources on the Server



POST is used to send data to the server to **create a new resource**.
It is **not safe** and not **idempotent**.

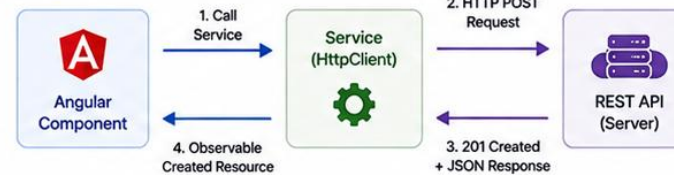
What is a POST Request?

POST sends data in the request body to the server to create a new resource. The server creates the resource and returns the created resource or its location.

Key Characteristics

- ✓ Not Safe – changes server state
- ✓ Not Idempotent – repeated requests create multiple resources
- ✓ Data is sent in the request body
- ✓ Used to create new resources
- ✓ Returns 201 Created on success

POST Request Flow



Common Use Cases

- Create a new employee
- Submit a form
- Register a new user
- Upload files/attachments
- Create orders or transactions

Service Method Example

```
// employee.service.ts
createEmployee(employee: Employee): Observable<Employee> {
  return this.http.post<Employee>(
    `${this.baseUrl}/employees`,
    employee,
    {
      headers: new HttpHeaders({
        'Content-Type': 'application/json'
      })
    }
  );
}
```

Example Component Usage

```
export class EmployeeFormComponent {
  constructor(private employeeService: EmployeeService) {}

  employee: Employee = {
    id: 0,
    firstName: '',
    lastName: '',
    email: '',
    department: ''
  };

  onSubmit(): void {
    this.employeeService.createEmployee(this.employee)
      .subscribe({
        next: (created) => {
          console.log('Employee created', created);
          this.router.navigate(['/employees']);
        },
        error: (err) => {
          console.error('Error creating employee', err);
        }
      });
  }
}
```

HTTP Request Example

POST Create Employee

POST https://api.example.com/api/employees

Request Headers

Content-Type: application/json
Accept: application/json

Request Body (JSON)

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "department": "Engineering",
  "position": "Developer"
}
```

Sample Success Response (201 Created)

```
{
  "id": 101,
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "department": "Engineering",
  "position": "Developer",
  "createdAt": "2025-05-18T10:30:00Z",
  "updatedAt": "2025-05-18T10:30:00Z"
}
```

Common Error Responses

- 400 Bad Request – Invalid input data
- 409 Conflict – Resource conflict
- 500 Internal Server Error – Server issue

Best Practices

- ✓ Validate data before sending
- ✓ Use typed interfaces for request and response
- ✓ Handle success and error responses properly
- ✓ Return the created resource or Location header
- ✓ Use 201 Created for successful creation

Tips

- Keep request body minimal and meaningful
- Avoid sending unnecessary fields
- Use environment variables for base URL
- Handle loading state while creating
- Show user feedback on success/failure

Typical Success Flow



Key Takeaway

POST requests create new resources on the server. Send data in the request body, handle the response, and update the UI accordingly.

PUT Requests

Replacing a Resource on the Server



PUT is used to update (replace) an existing resource on the server.
It is **not safe** but it is **idempotent**.



What is a PUT Request?

PUT replaces an entire resource at the target URI with the new representation provided in the request body.
If the resource does not exist, the server may create it.

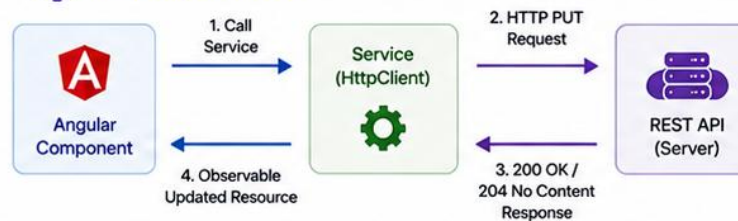


Key Characteristics

- ✓ **Not Safe** – changes server state
- ✓ **Idempotent** – repeated requests with the same data produce the same result
- ✓ Replaces the entire resource
- ✓ Data is sent in the request body
- ✓ Returns 200 OK (or 204 No Content) on success



PUT Request Flow



Common Use Cases

- Update an employee's details
- Replace user profile information
- Update configuration settings
- Replace entire documents/records
- Sync data from client to server

Service Method Example

```
// employee.service.ts
updateEmployee(id: number, employee: Employee): Observable<Employee> {
  return this.http.put<Employee>({
    url: `${this.baseUrl}/employees/${id}`,
    body: employee,
    headers: new HttpHeaders({
      'Content-Type': 'application/json'
    })
  });
}
```

Example Component Usage

```
export class EmployeeEditComponent {
  constructor(private employeeService: EmployeeService,
    private route: ActivatedRoute,
    private router: Router) {}

  employee: Employee = {} as Employee;

  onSubmit(): void {
    const id = this.route.snapshot.paramMap.get('id');
    if (id) {
      this.employeeService.updateEmployee(+id, this.employee)
        .subscribe({
          next: (updated) => {
            console.log('Employee updated', updated);
            this.router.navigate(['/employees']);
          },
          error: (err) => console.error('Update failed', err)
        });
    }
  }
}
```

HTTP Request Example

PUT Update Employee

PUT https://api.example.com/api/employees/101

Request Headers

Content-Type: application/json
Accept: application/json

Request Body (JSON)

```
{
  "id": 101,
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "department": "Engineering",
  "position": "Developer"
}
```

Sample Success Response (200 OK)

```
{
  "id": 101,
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "department": "Engineering",
  "position": "Developer",
  "updatedAt": "2025-05-18T10:45:21Z"
}
```

Common Error Responses

- 400 Bad Request – Invalid input data
- 404 Not Found – Resource does not exist
- 409 Conflict – Resource version mismatch
- 500 Internal Server Error – Server issue

Best Practices

- ✓ Always send the complete resource representation
- ✓ Use typed interfaces for requests and responses
- ✓ Validate data before sending
- ✓ Handle success and error responses properly
- ✓ Use 200 OK or 204 No Content for updates



Tips

- PUT is different from PATCH – PUT replaces the entire resource
- Ensure all required fields are included
- Use versioning or ETags for concurrency control
- Consider using PATCH for partial updates

Typical Update Flow



Key Takeaway

PUT requests update (replace) an existing resource completely. They are idempotent, so repeating the same request results in the same state on the server.

16 PATCH Requests

PATCH is used to apply **partial modifications** to an existing resource. It updates **only the fields provided** in the request.

Key Points

- ✓ **Partial Update**
Updates only the specified fields; other fields remain unchanged.
- ✓ **Not Idempotent by Default**
A PATCH request may not be idempotent unless designed to be.
- ✓ **Use for Partial Changes**
Ideal when the client wants to modify one or a few fields without sending the entire resource.
- ✓ **Content-Type: application/json**
Most commonly used with JSON payloads.
- ✓ **Common Use Cases**
Updating status, flags, counters, preferences, or any subset of fields.

Example

PATCH Request

```
PATCH /api/employees/101 HTTP/1.1
Host: api.example.com
Content-Type: application/json
Authorization: Bearer <token>
{
  "email": "john.doe@company.com",
  "status": "ACTIVE"
}
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "id": 101,
  "name": "John Doe",
  "email": "john.doe@company.com",
  "status": "ACTIVE",
  "department": "Engineering"
}
```



Partial Modification

Only the fields provided in the request are updated.



Efficient Updates

Reduces data transfer and avoids overwriting unchanged data.



Safer Than PUT

Does not require sending the entire resource representation.



Flexible

Supports dynamic and granular updates.



Best Practice

Use PATCH for partial updates and PUT for full replacements.



Key Takeaway: Use PATCH when you need to update specific fields of a resource. It improves efficiency and reduces the risk of overwriting other data.

/api/employees/101

PATCH

26 DELETE Requests

DELETE is used to **remove a resource** identified by its URI.
After a successful DELETE, the resource **no longer exists** on the server.

Key Points

- ✓ **Removes the Resource**
Deletes the resource identified by the URI.
- ✓ **Idempotent**
Multiple DELETE requests for the same resource have the same effect.
- ✓ **No Request Body**
DELETE requests typically do not include a body.
- ✓ **Appropriate Status Codes**
Use 200 OK, 204 No Content, or 404 Not Found based on the result.
- ✓ **Cascading Rules**
Related data may be deleted or restricted based on business rules and foreign key constraints.

Example

DELETE Request

```
DELETE /api/employees/101 HTTP/1.1
Host: api.examplee.com
Authorization: Bearer <token>
```

Successful Response (204 No Content)

```
HTTP/1.1 204 No Content
Content-Length: 0
```

Resource Not Found (404)

```
HTTP/1.1 404 Not Found
Content-Type: application/json
{
  "timestamp": "2025-05-18T10:30:00Z",
  "status": 404,
  "error": "Not Found",
  "message": "Employee with id 101 not found"
}
```



Common Use Cases

- Deleting an employee
- Removing a user account
- Deleting a record that is no longer needed



Best Practices

- Confirm deletion intent on the client side.
- Return meaningful status codes.
- Do not expose internal delete rules.
- Handle foreign key constraints properly.
- Implement soft delete if needed for audit/history.



Note

DELETE should be used carefully. Once deleted, the resource may be lost permanently unless soft delete or backup is implemented.



Key Takeaway: Use DELETE to remove resources when they are no longer needed. It is idempotent and should return the appropriate status code.

/api/employees/101

DELETE

REQUEST PARAMETERS

Path variables identify a resource; query parameters filter, sort, or page a collection.

PARAMETER TYPE	ANGULAR EXAMPLE
Path variable	<code>http.get<Customer>(`\${baseUrl}/\${customerId}`)</code> -- identifies exactly one resource.
Query parameter (single)	<code>new HttpParams().set('status', 'ACTIVE')</code> -- filters a collection.
Query parameter (multiple)	<code>params.set('page', 0).set('size', 20).set('sort', 'lastName')</code> -- pagination + sort together.
Optional parameter	Only <code>.set()</code> a param when the value is defined -- omit undefined filters entirely.
HttpParams immutability	Each <code>.set()</code> returns a new <code>HttpParams</code> instance -- the original is never mutated.

KEY TAKEAWAY `HttpParams` is immutable by design -- always chain or reassign the result of `.set()`, never expect it to mutate in place.

18 Request Parameters

Request parameters are used to send **additional information** to the server as part of the URL. They help **filter**, **search**, **sort**, or customize the response.

 Request parameters are passed in the URL after the ? as **key-value pairs**.

Key Points

-  **Passed in the URL**
Parameters are appended to the endpoint after the ? in **key=value** format.
-  **Used for Filtering & Searching**
Commonly used to filter results or search based on criteria.
-  **Optional or Required**
Some parameters are optional, while others may be required.
-  **Multiple Parameters**
Multiple parameters are separated by **&**.
-  **Type Conversion**
Parameter values are always strings in the URL and must be converted on the server.

Example Request

GET Request with Parameters

```
GET /api/employees?department=Engineering&page=1&size=10&sort=name,asc
Host: api.example.com
Authorization: Bearer <token>
```

`/api/employees?department=Engineering&page=1&size=10&sort=name,asc`

Endpoint Filter by department Page number Page size Sort by name (asc)

How the Server Receives It

```
@GetMapping("/api/employees")
public List<Employee> getEmployees(
    @RequestParam String department,
    @RequestParam(defaultValue = "1") int page,
    @RequestParam(defaultValue = "10") int size,
    @RequestParam(defaultValue = "name,asc") String sort
) {
    // logic
}
```

@RequestParam

- department → Filter value
- page → Page number
- size → Page size
- sort → Sorting order

Common Examples

```
GET /api/employees?status=ACTIVE
// Filter by status

GET /api/employees?dept=IT&status=ACTIVE
// Multiple filters

GET /api/employees?search=john
// Search by keyword

GET /api/employees?page=2&size=20
// Pagination

GET /api/employees?sort=createdAt,desc
// Sorting
```

Notes

- ✓ Use descriptive parameter names.
- ✓ Validate and sanitize parameter values.
- ✓ Avoid too many parameters in one request.
- ✓ Use sensible default values for optional params.

 **Key Takeaway:** Request parameters make your APIs flexible and powerful. Use them to filter, search, sort, and paginate data efficiently.

`/api/employees` ? `key1=value1` & `key2=value2` & `key3=value3`

REQUEST HEADERS

Headers carry metadata about the request -- authentication, content type, and correlation IDs.

HEADER	ROLE IN THIS COURSE'S API
Content-Type	application/json -- set automatically by HttpClient when the body is a plain object.
Authorization	Bearer <jwt> -- attached by an interceptor, never set manually inside a component.
Accept	application/json -- tells Spring Boot which representation the client wants back.
X-Correlation-Id	A per-request tracing ID, propagated end to end for log correlation across services.
HttpHeaders	Angular's immutable header builder, same .set()-returns-a-new-instance pattern as HttpParams.

KEY TAKEAWAY Authorization headers are attached by an interceptor, not hand-written in every service method -- that consistency is exactly why interceptors exist.

19 Request Headers

Request headers provide **metadata** about the request. They include details like **content type**, **authentication**, **client info**, and **preferred response formats**.



Headers are sent in the request before the optional body. They help the server understand how to process the request.



Key Points



Metadata

Headers send extra information about the request, not the data itself.



Common Uses

Authentication, content type, language preference, caching, and more.



Not Case-Sensitive

Header names are case-insensitive (e.g., content-type = Content-Type).



Key-Value Format

Each header is a key-value pair separated by a colon (:).



Optional but Powerful

Some headers are required, while others are optional depending on the use case.

Example Request

HTTP Request with Headers

```
POST /api/employees HTTP/1.1
Host: api.example.com
Content-Type: application/json
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
Accept-Language: en-US
User-Agent: PostmanRuntime/7.37.0
X-Request-ID: 8f3d2b45-7a2d-4c9b-9b9e-1e2f3a4b5c6d

{
  "name": "John Doe",
  "email": "john.doe@company.com",
  "department": "Engineering"
}
```

How the Server Uses Headers

- ✓ Identifies the client and its capabilities
- ✓ Validates authentication and authorization
- ✓ Determines how to parse the body
- ✓ Decides the response format and language
- ✓ Supports caching, logging, rate limiting, and more

Common Headers

Header	Purpose	Example Value
Host	Domain of the server	api.example.com
Content-Type	MIME type of request body	application/json
Accept	MIME type of response	application/json
Authorization	Authentication token	Bearer <token>
Accept-Language	Preferred language	en-US
User-Agent	Client/application info	PostmanRuntime/7.37.0
X-Request-ID	Custom request tracking	8f3d2b45-7a2...



Best Practices

- Always send Content-Type for requests with a body.
- Use Authorization header for secure APIs.
- Avoid sending sensitive data in custom headers.
- Keep headers relevant and avoid unnecessary headers.



Key Takeaway: Request headers carry important **metadata** that helps the server **understand**, **validate**, and **process** your request correctly.

Content-Type: application/json

Accept: application/json

Authorization: Bearer <token>

WORKING WITH TYPED RESPONSES

A generic type argument on every HttpClient call turns 'some JSON' into a compile-time-checked shape.



Generic Type Parameter

`http.get<Customer[]>(url)` tells TypeScript the Observable emits a Customer array -- autocomplete follows from there.



No Runtime Guarantee

TypeScript types erase at compile time -- a mismatched backend response still 'succeeds' with undefined fields.



Structural, Not Nominal

Any object with matching field names satisfies the interface -- Angular doesn't care what class the backend used.

KEY TAKEAWAY Typing an HttpClient call is a promise to the compiler, not a guarantee from the network -- keeping backend DTOs and frontend interfaces in sync is a discipline, not something the type system enforces for you.

20 Working with Typed Responses

Typed responses in Angular provide **compile-time safety** and **better developer experience**. Use **generics** with HttpClient methods to define the expected response type.

i Strongly typed responses help you avoid runtime errors and give you **IntelliSense** in your IDE.

Key Benefits



Compile-Time Safety

Catches type mismatches during development instead of runtime.



Better Developer Experience

Full IntelliSense and auto-completion in IDE.



Self-Documenting Code

The expected response shape is clear in the code.



Easier Refactoring

TypeScript helps you update usages across the application.



Reliable Data Handling

Reduces accidental typos and invalid property access.

Example: Typed GET Request

employee.service.ts

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Employee } from '../models/employee.model';

@Injectable({ providedIn: 'root' })
export class EmployeeService {
  private apiUrl = 'https://api.example.com/api/employees';

  constructor(private http: HttpClient) {}

  // Typed response using <Employee[]>
  getEmployees(): Observable<Employee[]> {
    return this.http.get<Employee[]>(this.apiUrl);
  }
}
```

Example Model

employee.model.ts

```
export interface Employee {
  id: number;
  firstName: string;
  lastName: string;
  email: string;
  department: string;
  salary: number;
  active: boolean;
}
```

Maps
↔

Response JSON (from API)

employees response

```
[
  {
    "id": 101,
    "firstName": "John",
    "lastName": "Doe",
    "email": "john.doe@company.com",
    "department": "Engineering",
    "salary": 75000,
    "active": true
  },
  {
    "id": 102, "firstName": "Jane", ...
  }
]
```

Example: Using the Service in a Component

employee-list.component.ts

```
import { Component, OnInit } from '@angular/core';
import { EmployeeService } from '../services/employee.service';
import { Employee } from '../models/employee.model';

@Component({ /* ... */ })
export class EmployeeListComponent implements OnInit {
  employees: Employee[] = [];
  loading = false;
  error: string | null = null;

  constructor(private employeeService: EmployeeService) {}

  ngOnInit(): void {
    this.loading = true;
    this.employeeService.getEmployees().subscribe({
      next: (data: Employee[]) => {
        this.employees = data;
        this.loading = false;
      },
      error: (err) => {
        this.error = err.message || 'Failed to load employees';
        this.loading = false;
      }
    });
  }
}
```

HttpClient Syntax

```
this.http.get<T>(url, options?): Observable<T>
```



Key Takeaway: Always use typed responses with HttpClient. It improves code quality, prevents runtime errors, and enhances maintainability.

TYPESCRIPT INTERFACES FOR API MODELS

One interface per DTO shape -- request models and response models are kept separate, not shared.



Response Interface

interface Customer mirrors the backend's response DTO -- id, firstName, lastName, email, createdAt.



Request Interface

interface CreateCustomerRequest omits id and createdAt -- fields the client never sends, only the server generates.



One File per Resource

models/customer.model.ts holds every Customer-related interface, imported wherever the resource is used.

KEY TAKEAWAY Never reuse one interface for both what you send and what you receive -- a create request and its response have genuinely different shapes.

21 TypeScript Interfaces for API Models

TypeScript interfaces define the **shape of data** exchanged with the API. They ensure **type safety**, **better IntelliSense**, and **consistent data handling** across the application.



Why Use Interfaces?

They describe the structure of API request and response data, helping the compiler catch errors early.



Key Benefits



Type Safety

Prevents type mismatches and runtime errors.



Better Developer Experience

Provides IntelliSense, auto-completion, and inline documentation.



Consistent Contract

Interfaces act as a contract between frontend and backend.



Easier Refactoring

Changes in the model are reflected across the application.



Reusability

Interfaces can be reused across services, components, and modules.

Example: API Response Model

JSON Response from API

```
{
  "id": 101,
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@company.com",
  "department": "Engineering",
  "salary": 75000,
  "isActive": true,
  "roles": ["DEVELOPER", "ADMIN"],
  "createdAt": "2025-05-18T10:30:00Z"
}
```

TypeScript Interface

```
export interface Employee {
  id: number;
  firstName: string;
  lastName: string;
  email: string;
  department: string;
  salary: number;
  isActive: boolean;
  roles: string[];
  createdAt: string;
}
```

Example: Using Interface in Angular Service

employee.service.ts

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Employee } from '../models/employee.model';

@Injectable({ providedIn: 'root' })
export class EmployeeService {
  private apiUrl = 'https://api.example.com/api/employees';

  constructor(private http: HttpClient) {}

  getEmployees(): Observable<Employee[]> {
    return this.http.get<Employee[]>(this.apiUrl);
  }
}
```

What Happens Here?

- ✓ The API returns JSON data.
- ✓ Angular converts it to Employee[] based on the interface.
- ✓ The compiler ensures you access only valid properties.
- ✓ IntelliSense suggests available fields in the IDE.



Common Patterns

Optional Properties

```
// ? makes the property optional
export interface UpdateEmployeeRequest {
  id: number;
  email?: string;
  department?: string;
}
```

Readonly Properties

```
export interface AuditInfo {
  readonly createdBy: string;
  readonly createdAt: string;
}
```

Nested Interfaces

```
export interface Department {
  id: number;
  name: string;
}
export interface EmployeeWithDept extends Employee {
  department: Department;
}
```

Best Practices

- ✓ Keep interfaces aligned with backend DTOs.
- ✓ Use meaningful and consistent naming.
- ✓ Prefer interfaces over types for object shapes.
- ✓ Reuse interfaces across the application.



Key Takeaway: Use TypeScript interfaces to model API data. They provide type safety, improve productivity, and keep your frontend and backend in sync.

API JSON Response



TypeScript Interface



Typed Data in Angular

DTO ALIGNMENT ACROSS FRONTEND AND BACKEND

Field-by-field, the Angular interface and the Spring Boot DTO must describe the exact same JSON.

SPRING BOOT DTO FIELD	ANGULAR INTERFACE FIELD
String customerId	customerId: string
BigDecimal lifetimeValue	lifetimeValue: number
LocalDateTime createdAt	createdAt: string // ISO-8601, not Date
List<String> tags	tags: string[]
CustomerStatus status (enum)	status: 'ACTIVE' 'INACTIVE' 'PROSPECT' // string literal union

KEY TAKEAWAY A backend enum becomes a TypeScript string literal union, not a TypeScript enum -- that keeps the JSON wire value and the Angular type in exact agreement.

TRY IT YOURSELF — EXERCISE 2: ALIGN TYPED MODELS TO THE API

Reinforces: typed responses, TypeScript interfaces for API models, and DTO alignment across frontend and backend as you just covered.

STEPS (notes/lab35-typed-models.md)

Step	What To Do
1 — Customer Interface	Write the Customer interface fields and types, matching the Java DTO.
2 — Typed Call	Show the generic type argument on the HttpClient get call.
3 — Drift Risk	Name one field where the Java and TypeScript types could silently disagree.
4 — Paged Response	Define the wrapper type the list endpoint returns.

Model alignment

```
interface Customer { id: string; name: string; email: string;
                    status: 'ACTIVE'|'PROSPECT'; createdAt: string }
http.get<Page<Customer>>(url)    // Page{content,page,size,totalElements}
```

TRY IT NOW — Complete Exercise 2 before continuing: `exercises/exercise-02-typed-models.md` (workspace: `examples/module-35-exercises`).

22

DTO Alignment Across Frontend and Backend

DTO (Data Transfer Object) alignment ensures that the **data structure** used by the backend API **matches exactly** with the models used in the **Angular frontend**.



Why Alignment Matters

- Prevents runtime errors and mapping issues
- Ensures consistent data exchange
- Improves developer productivity and maintenance

1 Backend DTO (Java)

```
public class EmployeeDTO {
    private Long id;
    private String firstName;
    private String lastName;
    private String email;
    private String department;
    private BigDecimal salary;
    private Boolean isActive;
    private LocalDateTime createdAt;

    // getters and setters
}
```



Serialized
by Backend

2 API Response (JSON)

```
[
  {
    "id": 101,
    "firstName": "John",
    "lastName": "Doe",
    "email": "john.doe@company.com",
    "department": "Engineering",
    "salary": 75000.00,
    "isActive": true,
    "createdAt": "2025-05-18T10:30:00Z"
  }
]
```



Consumed
by Frontend

3 Frontend Model (TypeScript Interface)

```
export interface EmployeeDTO {
    id: number;
    firstName: string;
    lastName: string;
    email: string;
    department: string;
    salary: number;
    isActive: boolean;
    createdAt: string; // ISO 8601 date
}
```



Best Practices

- Use DTOs instead of Entities in APIs
- Keep fields relevant to the client
- Use proper data types (not String for numbers)
- Follow consistent naming conventions (camelCase)



Key Points

- JSON keys must match DTO field names
- Data types must be compatible
- Date format should be consistent (ISO 8601)
- Avoid extra fields that the frontend does not need



Best Practices

- Keep interface in sync with backend DTO
- Use same property names and compatible types
- Use meaningful TypeScript types
- Reuse interfaces across services and components



Alignment Checklist

- ✓ Same Property Names
- ✓ Compatible Data Types
- ✓ Consistent Date Formats
- ✓ No Unnecessary Extra Fields
- ✓ Clear Handling of Nulls
- ✓ Updated When Backend Changes

Common Misalignment Example

Backend JSON

```
• "first_name": "John"
• "salary": "75000" // as string
• "created_at": "..."
```

Frontend Interface

```
• firstName: string
• salary: number
• createdAt: string
```

Issues

- Different property names
- Type mismatch
- Breaks mapping
- Causes runtime errors



Key Takeaway: Keep your DTOs and frontend interfaces aligned. It's the foundation of smooth, reliable API communication.

Backend DTO (Java)



JSON Response



Frontend Interface (TypeScript)

OBSERVABLE-BASED HTTP REQUESTS

Every HttpClient call returns a cold Observable -- nothing is sent until something subscribes.



Cold, Not Hot

The HTTP request only fires when `.subscribe()` is called (directly or via `AsyncPipe`) -- creating it alone does nothing.



Emits Once, Then Completes

A typical `HttpClient Observable` emits one value, the response, then completes -- not an ongoing stream.



Must Be Cleaned Up

A manual `.subscribe()` outside `AsyncPipe` should be unsubscribed in `ngOnDestroy` to avoid a destroyed component updating.

KEY TAKEAWAY If nothing happens after calling an `ApiService` method, the first thing to check is whether the returned `Observable` was ever actually subscribed to.

23 Observable-Based HTTP Requests

Angular's HttpClient methods return **Observable<T>**. Observables represent **asynchronous** HTTP operations that **emit a response** when it's ready.



Key Idea

HttpClient doesn't return the data directly. It returns an **Observable** that will emit the data in the future.



Key Benefits



Asynchronous by Default

Non-blocking requests that keep the UI responsive.



Composable

Use RxJS operators (map, filter, retry, etc.) to transform and handle data.



Cancelable

Unsubscribe to cancel in-flight HTTP requests.



Consistent API

All HttpClient methods return Observable, promoting a consistent programming model.



Error Handling

Handle errors gracefully using RxJS operators like catchError.

Example: Angular Service

employee.service.ts

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Employee } from '../models/employee.model';

@Injectable({ providedIn: 'root' })
export class EmployeeService {
  private apiUrl = 'https://api.example.com/api/employees';

  constructor(private http: HttpClient) {}

  // GET: Observable<Employee[]>
  getEmployees(): Observable<Employee[]> {
    return this.http.get<Employee[]>(this.apiUrl);
  }

  // GET by Id: Observable<Employee>
  getEmployeeById(id: number): Observable<Employee> {
    return this.http.get<Employee>(`${this.apiUrl}/${id}`);
  }
}
```

What Happens Behind the Scenes?

- 1 Component calls a service method. The method returns an Observable<T>.
- 2 HttpClient sends the HTTP request. Request is sent asynchronously.
- 3 Server processes and returns a response. Response is received in the background.
- 4 Observable emits the response data. The data is pushed to the subscriber.
- 5 Subscriber receives the data. Component updates the UI.

HttpClient Methods (All return Observable<T>)

Method	Description	Example
get<T>()	Retrieves data (GET)	get<Employee[]>(url)
post<T>()	Creates data (POST)	post<Employee>(url, body)
put<T>()	Replaces data (PUT)	put<Employee>(url, body)
patch<T>()	Partially updates data (PATCH)	patch<Employee>(url, body)
delete<T>()	Deletes data (DELETE)	delete<void>(url)

Example: GET Request Returns Observable

```
// In Service
getEmployees(): Observable<Employee[]> {
  return this.http.get<Employee[]>(this.apiUrl);
}

// In Component
this.employeeService.getEmployees().subscribe(data => {
  this.employees = data; // handle success
}, error => {
  this.error = error; // handle error
});
```

RxJS Observable Flow



Observable

Represents a future value



Operators

Transform, filter, map, etc.



Subscribe

Start listening to values



Next

Receives emitted to values



Error

Handles errors if any



Complete

Signals completion of the stream

Common RxJS Operators with HTTP

map()	Transforms the response data.
tap()	Performs side effects (logging, debugging).
catchError()	Handles errors and returns fallback data.
retry()	Retries the request on failure.
finalize()	Executes logic when the observable completes or errors.

Example:

```
this.http.get<Employee[]>(this.apiUrl).pipe(
  map(res => res.filter(e => e.isActive)),
  tap(data => console.log('Employees:', data)),
  catchError(err => of([])),
  finalize(() => console.log('Completed'))
).subscribe(data => this.employees = data);
```



Key Takeaway: HttpClient returns **Observable<T>**, not the data directly. Subscribe to the observable to get the data and handle success/error/complete events.

HttpClient Method
(Returns Observable)

Subscribe()
(Start Listening)

Receive Data
(Next)

Handle Error
(Error)

Complete
(Complete)

SUBSCRIBING TO API RESPONSES

subscribe({ next, error, complete }) is the manual way to consume an HttpClient Observable in TypeScript code.



next

Runs once with the successful response value
-- update component state here.



error

Runs if the request fails (network error or
non-2xx status) -- update error state here.



complete

Runs after next, only on success -- rarely
needed for a single-emission call.

KEY TAKEAWAY Always supply an error callback -- an HttpClient Observable with no error handler turns a failed request into a silent, unhandled rejection in the browser console.



Subscribing to API Responses



Angular HttpClient returns **Observables**. You must subscribe to them to execute the HTTP request and receive the response.

Key Points



HttpClient methods return **Observables**.



The API call is lazy and executes only when subscribed.



Handle **success**, **error**, and **completion**.



Unsubscribe to prevent memory leaks.

employee.service.ts

```
1  getEmployees(): Observable<Employee[]> {
2    return this.http.get<Employee[]>('/api/employees');
3  }
4
5  loadEmployees() {
6    this.employeeService.getEmployees().subscribe({
7      next: (employees) => {
8        this.employees = employees;
9        this.isLoading = false;
10     },
11     error: (err) => {
12       console.error('Error loading employees', err);
13       this.errorMessage = 'Failed to load employees.';
14     }
15   });
16 }
```



Best Practices



Subscribe in **components**, not in services.



Unsubscribe in **ngOnDestroy** or use **async pipe**.



Use **retry()** for transient failures.



Show **user-friendly messages** on errors.

How It Works



Component Makes a Call



HttpClient Returns an **Observable**



subscribe() Is Called



HTTP Request Is Sent



Backend Processes the Request



Response Received (**next**)



Error Received (**error**)



Observable **Completes**

ASYNCPipe

customers\$ / async subscribes and unsubscribes automatically inside the template -- no manual cleanup needed.



customers\$ | async

The pipe subscribes to the Observable when the template renders and re-renders whenever a new value is emitted.



Automatic Unsubscription

AsyncPipe unsubscribes automatically when the component is destroyed -- no ngOnDestroy boilerplate required.



No Manual .subscribe()

The component class holds an Observable field, never a plain array field populated by a manual subscribe call.

KEY TAKEAWAY Prefer customers\$ | async over a manual subscribe() in almost every template-display scenario -- it eliminates an entire class of memory-leak bugs by construction.



AsyncPipe

AsyncPipe is an Angular built-in pipe that subscribes to an Observable or Promise and **automatically returns the latest value**. It simplifies templates by handling subscription, emission, and unsubscription for you.

★ Key Benefits

-  Automatically **subscribes** to Observables / Promises.
-  Automatically **updates** the template when new values are emitted.
-  Automatically **unsubscribes** when the component is destroyed (prevents memory leaks).
-  Keeps components **clean** and focused on logic.

TS employee.service.ts

```
1 getEmployees(): Observable<Employee[]> {  
2   return this.http.get<Employee[]>('/api/employees');  
3 }  
4  
5 employees$ = this.getEmployees(); // Observable<Employee[]>  
6  
7
```

HTML employee-list.component.html

```
1 <div class="card">  
2   <h3>Employees</h3>  
3   <div *ngIf="employees$ | async as employees; else loading">  
4     <ul class="list-group">  
5       <li class="list-group-item" *ngFor="let emp of employees">  
6         {{ emp.name }} - {{ emp.department }}  
7       </li>  
8     </ul>  
9   </div>  
10
```

✓ Best Practices

-  Use **AsyncPipe** in templates instead of manual **subscribe()**.
-  Ideal for displaying data directly in the UI.
-  Combine with **loading**, **error**, and **empty states**.
-  Let **AsyncPipe** manage the subscription lifecycle.

Example Output



Without AsyncPipe (Not Recommended)

```
1 employees: Employee[] = [];  
2 ngOnInit() {  
3   this.employeeService.getEmployees().subscribe(data => {  
4     this.employees = data;  
5   });  
6 }
```



Manual **subscribe()** requires unsubscription in **ngOnDestroy()** to avoid memory leaks.

How It Works



MAPPING API RESPONSES

The pipe(map(...)) operator reshapes a response before it ever reaches the component.



pipe(map(...))

map(page => page.content) extracts just the array a component needs from a paginated response envelope.



Derived Fields

map() can compute a display-only field (e.g. fullName) without asking the backend to add it to the DTO.



Keep It in the Service

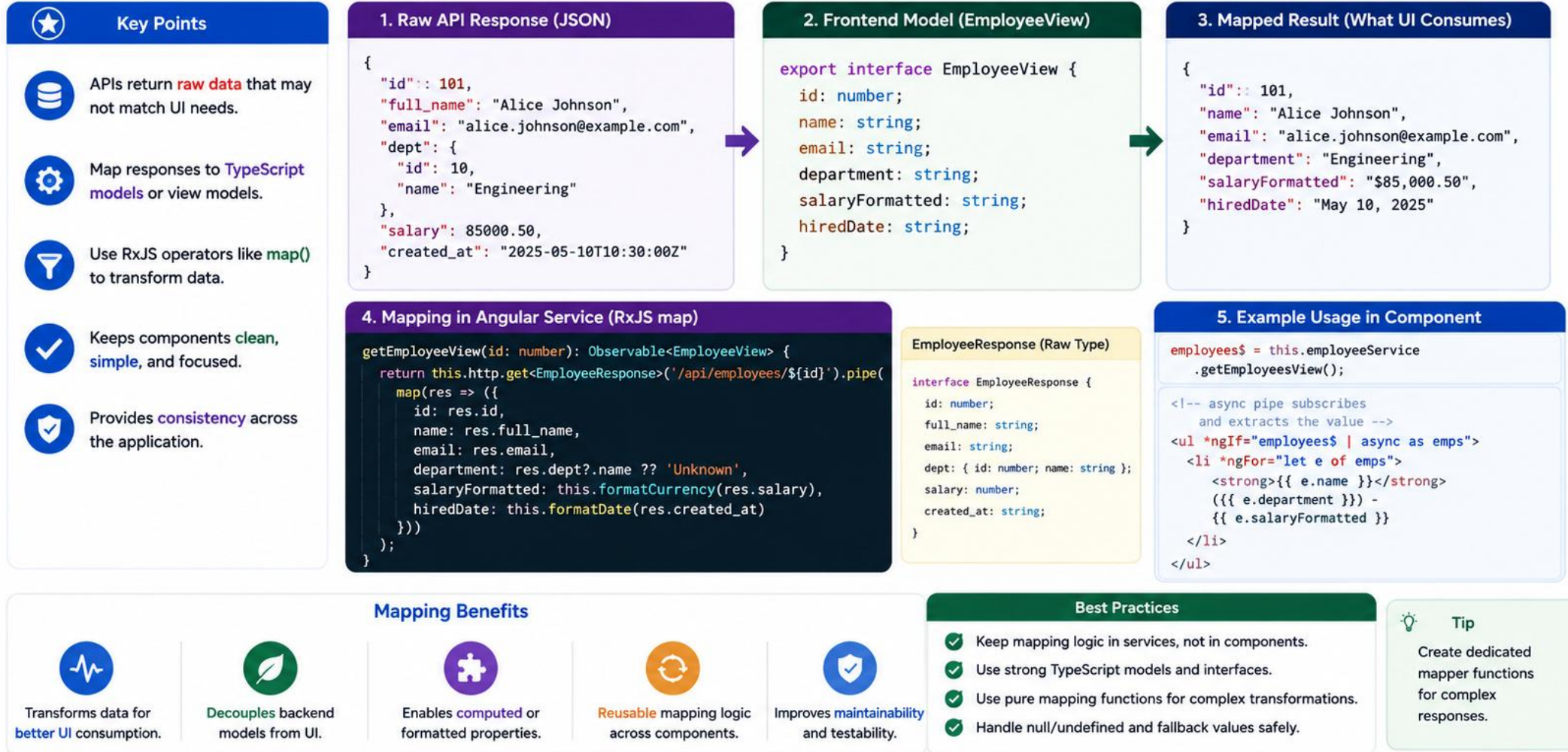
Response shaping belongs in the ApiService, so every component consuming it sees the same, already-clean shape.

KEY TAKEAWAY Reshape the response once, inside the service, with pipe(map(...)) -- don't make every component that consumes the data repeat the same transformation.



Mapping API Responses

API responses often need to be **transformed** or mapped into **UI-friendly models** before we use them in components.



UI STATES: LOADING, EMPTY, AND ERROR

Every screen that calls the API has at least four states -- loading, empty, error, and success.



Loading State

Shown from the moment a request is sent until next or error fires -- a spinner or skeleton, never a blank screen.



Empty State

Shown when the request succeeds but returns zero results -- a clear message, not an empty-looking bug.



Error State

Shown when the request fails -- a human-readable message, never a raw `HttpResponse` dumped to the screen.

KEY TAKEAWAY A component with only a 'success' state is an unfinished component -- loading, empty, and error are not edge cases, they are required states.



Loading States

Loading states inform users that data is being fetched from the server. Good loading experiences **improve perceived performance** and prevent confusion.



Key Points



Show a **loading state** during async operations.



Prevent user actions that depend on data.



Use **loaders, skeletons, or progress indicators**.



Hide loading indicators once **data is ready**.



Provide a **smooth** user experience.

TS Component (TypeScript)

```
1 export class EmployeeListComponent implements OnInit {
2   employees: Employee[] = [];
3   isLoading = true;
4   error: string | null = null;
5
6   constructor(private employeeService: EmployeeService) {}
7
8   ngOnInit(): void {
9     this.employeeService.getEmployees().subscribe({
10      next: (data) => { this.employees = data; this.isLoading = false; },
11      error: (err) => { this.error = 'Failed to load employees.';
12        this.isLoading = false; }
13    });
14  }
15 }
```

Template (HTML)

```
<!-- Loading State -->
<div *ngIf="isLoading" class="loading-container">
  <app-spinner></app-spinner>
  <p>Loading employees...</p>
</div>

<!-- Error State -->
<div *ngIf="error" class="error-container">
  <app-error-message [message]="error"></app-error-message>
</div>

<!-- Data State -->
<div *ngIf="!isLoading && !error">
  <app-employee-list [employees]="employees"></app-employee-list>
</div>
```

UI Examples

Spinner



Loading employees...

Progress Bar



Loading employees...

Skeleton Loader



Overlay Loader

Name	Department
Alice Johnson	Engineering
Bob Smith	Finance
Charlie Brown	HR

Best Practices



Show loading **immediately** on user action.



Keep loaders **lightweight** and non-blocking.



Maintain visual **consistency** across the application.



Ensure loaders are **accessible** (ARIA labels / roles).

Loading State Flow



User Action



Set isLoading = true



API Request Sent



Server Processing



Data Received



Update Data



Set isLoading = false



Empty States

Empty states are shown when there is no data to display. They help set **clear expectations** and **guide users** on what to do next.



Key Points



Display when API returns **empty arrays** or null.



Communicate clearly there is no data.



Provide helpful **next steps**.



Make empty states visually distinct but consistent.



Improve user experience and reduce confusion.

TS Component (TypeScript)

```
1 employees: Employee[] = [];  
2 isLoading = false;  
3 error: string | null = null;  
4  
5 loadEmployees(): void {  
6   this.isLoading = true;  
7   this.employeeService.getEmployees().subscribe({  
8     next: (data) => { this.employees = data; this.isLoading = false; },  
9     error: (err) => { this.error = 'Failed to load employees.';  
       this.isLoading = false; }  
    });  
9 }
```



Template (HTML)

```
<!-- Loading State -->  
<app-spinner *ngIf="isLoading"></app-spinner>  
  
<!-- Error State -->  
<app-error-message *ngIf="error" [message]="error"></app-error-message>  
  
<!-- Empty State -->  
<div *ngIf="!isLoading && !error && employees.length === 0"  
  class="empty-state">  
  <app-empty-state  
    title="No Employees Found"  
    message="There are no employees to display."  
    icon="👤"  
    primaryActionLabel="Add Employee"  
    (primaryAction)="navigateToAddEmployee()" >  
  </app-empty-state>  
</div>  
  
<!-- Data State -->  
<app-employee-list *ngIf="employees.length > 0" [employees]="employees"></app-employee-list>
```

Empty State UI Examples

No Data Available



No items found.
There is no data to display.

Refresh

No Search Results



No results found for
"developer".

Clear Search

No Employees



No employees found.
Get started by adding your
first employee.

Add Employee

No Notifications



You're all caught up!
No new notifications.

View All



Best Practices



Use clear and **friendly** messages.



Suggest the **next best** action.



Keep design **consistent** across the app.



Ensure accessibility
(ARIA labels, roles, contrast).



Optimize for all screen sizes.

Empty State Flow



User opens page



API request starts
(isLoading = true)



API responds



Response is empty
(employees.length === 0)



Show empty state
with next action



User takes action
(Add / Create)



Data loads and
is displayed



Error States

Error states inform users that something went wrong and help them **understand** what happened and how to **recover**.



Key Points



Handle HTTP errors gracefully.



Show user-friendly error messages.



Provide actions like **Retry** or **Go Back**.



Log technical details for **debugging**.



Keep error information **secure** and concise.

TS Angular Service (TypeScript)

```
1 getEmployees(): Observable<Employee[]> {
2   return this.http.get<Employee[]>('/api/employees').pipe(
3     catchError((error: HttpResponse) => {
4       let userMessage = 'Failed to load employees.';
5       if (error.status === 0) {
6         userMessage = 'Network error. Please check your connection.';
7       } else if (error.status === 404) {
8         userMessage = 'No employees found.';
9       }
10      return throwError(() => new Error(userMessage));
11    })
12  );
13 }
```

TS Error Handling in Component

```
1 this.employeeService.getEmployees().subscribe({
2   next: (data) => {
3     this.employees = data;
4     this.errorMessage = null;
5   },
6   error: (err: Error) => {
7     this.errorMessage = err.message;
8     this.isLoading = false;
9   },
10  complete: () => this.isLoading = false
11 });
```



Best Practices



Show **meaningful** messages to end users.



Allow users to **retry** or take action.



Log complete error details on the **server**.



Avoid exposing **technical** details to users.



Maintain **consistency** across the application.

Common HTTP Error Scenarios

400 Bad Request



Invalid request.
Check your input.

Fix Input

401 Unauthorized



You are not authorized.
Please login again.

Login

403 Forbidden



You do not have permission.

Go Back

404 Not Found



The requested resource was not found.

Go to Home

500 Server Error



Something went wrong on our end.

Try Again

Example UI



Unable to load employees

Network error. Please check your connection and try again.

Retry

Go Home

Error State Flow



User triggers an action



API Request Sent



Error Occurs



Error Response Received



Error is handled in Service/Component



User-friendly message is displayed



User takes action (Retry / Go Back)



App recovers or navigates safely

TRY IT YOURSELF — EXERCISE 3: RENDER EVERY UI STATE

Reinforces: observable-based requests, subscribing, AsyncPipe, response mapping, and the loading/empty/error slides you just covered.

STEPS (notes/lab35-ui-states.md)

Step	What To Do
1 — Async Pipe	Say what the async pipe does that a manual subscribe does not.
2 — Three States	Describe the template branches for loading, empty, and error.
3 — Cold Reminder	Say why the request never fires if nothing subscribes.
4 — Map Step	Give one transformation applied between the response and the view.

Template state branches

```
@if (loading())      spinner
@else if (error())   banner + retry
@else if (!rows().length) 'No customers match' @else list
```

TRY IT NOW — Complete Exercise 3 before continuing: `exercises/exercise-03-ui-state-plan.md` (workspace: `examples/module-35-exercises`).



Error States

Error states inform users that something went wrong and help them **understand** what happened and how to **recover**.



Key Points



Handle HTTP errors **gracefully**.



Show **user-friendly** error messages.



Provide actions like **Retry** or **Go Back**.



Log technical details for **debugging**.



Keep error information **secure** and concise.

TS Angular Service (TypeScript)

```
1 getEmployees(): Observable<Employee[]> {
2   return this.http.get<Employee[]>('/api/employees').pipe(
3     catchError((error: HttpResponse) => {
4       let userMessage = 'Failed to load employees.';
5       if (error.status === 0) {
6         userMessage = 'Network error. Please check your connection.';
7       } else if (error.status === 404) {
8         userMessage = 'No employees found.';
9       }
10      return throwError(() => new Error(userMessage));
11    })
12  );
13 }
```

TS Error Handling in Component

```
1 this.employeeService.getEmployees().subscribe({
2   next: (data) => {
3     this.employees = data;
4     this.errorMessage = null;
5   },
6   error: (err: Error) => {
7     this.errorMessage = err.message;
8     this.isLoading = false;
9   },
10  },
11  complete: () => this.isLoading = false
12 });
```



Best Practices



Show **meaningful** messages to end users.



Allow users to **retry** or take action.



Log complete error details on the **server**.



Avoid exposing **technical** details to users.



Maintain **consistency** across the application.

Common HTTP Error Scenarios

400 Bad Request



Invalid request.
Check your input.

Fix Input

401 Unauthorized



You are not authorized.
Please login again.

Login

403 Forbidden



You do not have
permission.

Go Back

404 Not Found



The requested resource
was not found.

Go to Home

500 Server Error



Something went wrong
on our end.

Try Again

Example UI



Unable to load employees

Network error. Please check your connection
and try again.

Retry

Go Home

Error State Flow



User triggers
an action



API Request
Sent



Error Occurs



Error Response
Received



Error is handled in
Service/Component



User-friendly message
is displayed



User takes action
(Retry / Go Back)



App recovers or
navigates safely

HANDLING HTTP ERRORS

catchError() intercepts a failed Observable before it reaches the component's error callback.

STATUS CODE	TYPICAL ANGULAR HANDLING
400 Bad Request	Show field-level validation errors parsed from the Problem Details response body.
401 Unauthorized	Redirect to login -- the token is missing, invalid, or expired.
403 Forbidden	Show 'not permitted' messaging -- the user is authenticated but lacks the role.
404 Not Found	Show a not-found state -- do not treat it as a generic error banner.
500 Internal Server Error	Show a generic 'something went wrong, try again' message -- never expose details.

KEY TAKEAWAY Map each status code to a specific UI response -- a single generic 'Error!' banner for every failure is a worse experience than no error handling at all.



Handling HTTP Errors

HTTP requests can fail for many reasons. Handle errors **gracefully**, show **meaningful messages**, and keep the user **informed**.



Key Points



Use **catchError()** to intercept and handle HTTP errors.



Distinguish between client (4xx) and server (5xx) errors.



Show **user-friendly** messages, not raw error details.



Log **technical details** for debugging and monitoring.



Provide actions like **Retry**, **Go Back**, or **Contact Support**.

TS Angular Service (TypeScript)

```
1 getEmployees(): Observable<Employee[]> {
2   return this.http.get<Employee[]>('/api/employees').pipe(
3     catchError((error: HttpResponse) => {
4       let userMessage = 'Something went wrong.';
5
6       if (error.error instanceof ErrorEvent) {
7         // Client-side error (network, CORS, etc.)
8         userMessage = 'Network error. Please check your connection.';
9       } else {
10        // Server-side error
11        if (error.status === 404) userMessage = 'No employees found.';
12        else if (error.status === 401) userMessage = 'Unauthorized.';
13        else if (error.status >= 500)
14          userMessage = 'Server error. Please try again later.';
15      }
16      console.error('HTTP Error:', error); // Log for debugging
17      return throwError(() => new Error(userMessage));
18    })
19  );
20 }
```

Error Handling Flow



Request Sent



Error Occurs



catchError()
Intercepts



Show Friendly
Message



User Takes
Action

Error Sources



Network
Issues



Authentication
Failed (401)



Forbidden
(403)



Not Found
(404)



Server Error
(5xx)

Common HTTP Error Status Codes

Status Code	Type	Meaning	Example
400	Client Error	Bad Request	Invalid input
401	Client Error	Unauthorized	Login required
403	Client Error	Forbidden	No permission
404	Client Error	Not Found	Resource not found
409	Client Error	Conflict	Duplicate resource
500+	Server Error	Server Error	Server-side issue

UI Examples

404



No Employees Found

We couldn't find any employees.

[Go to Add Employee](#)

401



Unauthorized

Please login to continue.

[Login](#)

500



Server Error

Something went wrong on our end.
Please try again later.

[Retry](#)

Best Practices

- ✓ Always handle errors in the service layer.
- ✓ Return user-friendly messages.
- ✓ Log errors for diagnostics and monitoring.
- ✓ Use a global error handler for consistency.
- ✓ Provide clear actions for the user.
- ✓ Do not expose sensitive server details.
- ✓ Test error scenarios (4xx and 5xx).

RETRY CONSIDERATIONS

Only idempotent requests should ever be retried automatically -- and only for transient failures.



retry({ count, delay })

The RxJS retry operator resubmits a failed request a bounded number of times before giving up.



Safe: GET, PUT, DELETE

Idempotent verbs can be retried without risking duplicate side effects on the backend.



Unsafe: POST

Retrying a failed POST can create duplicate records -- never retry it silently without an idempotency key.

KEY TAKEAWAY A bounded retry with a short delay handles a flaky network gracefully -- an unbounded or naive retry can turn one dropped connection into a hammering loop.



Retry Considerations

Retries can improve resilience against transient failures, but they must be **used carefully** to avoid duplicates, performance issues, and poor user experience.



Key Points



Retry only for **transient failures** (network glitches, timeouts, 5xx).



Do not retry on client errors (4xx) except specific cases (e.g., 429).



Use **backoff strategies** to avoid overwhelming the server.



Limit the number of **retries** and total time.



Ensure **idempotency** for safe retries on write operations.

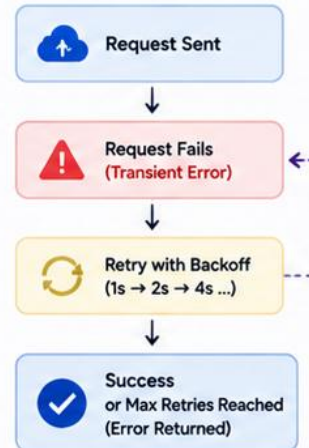
TS Angular Service (TypeScript + RxJS)

```
getEmployees(): Observable<Employee[]> {  
  return this.http.get<Employee[]>('/api/employees').pipe(  
    retryWhen(errors => errors.pipe(  
      scan((attempt, error) => {  
        if (attempt >= 3) {  
          throw error; // stop after 3 retries  
        }  
        return attempt + 1;  
      }, 0),  
      delayWhen(attempt => timer(this.getDelay(attempt))),  
      tap(attempt => console.warn('Retry attempt ${attempt}'))  
    )),  
    catchError(err => throwError(() => err))  
  );  
}  
  
private getDelay(attempt: number): number {  
  const baseDelay = 1000; // 1s  
  const maxDelay = 10000; // 10s  
  return Math.min(baseDelay * Math.pow(2, attempt - 1), maxDelay);  
}
```

What this does:

- ✓ Retries up to 3 times for transient errors.
- ✓ Uses exponential backoff.
- ✓ Logs each retry attempt.

Retry Strategy Flow



Best Practices



Retry only **transient failures** (network issues, timeouts, 5xx).



Do not retry **non-idempotent** operations unless safe.



Use **exponential backoff with jitter** to reduce load spikes.



Set **reasonable limits** on retries and total timeout.



Provide **user feedback** if retries take too long.



Log **retry attempts** for monitoring and debugging.



Ensure **idempotency keys** for POST/PUT if retries are possible.



When in Doubt

Start with a small number of retries (2-3) and monitor. Tune based on real-world behavior.



Retryable Scenarios (Examples)



- Network timeouts / connectivity issues
- HTTP 5xx (500, 502, 503, 504)
- Rate limited (429) – after 'Retry-After'
- Temporary DNS failures
- Service unavailable / overloaded

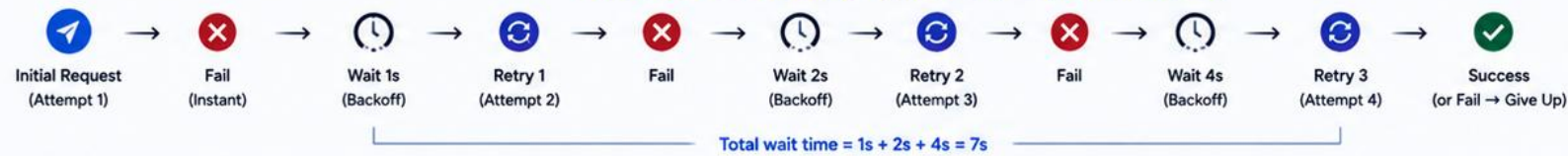


Non-Retryable Scenarios (Examples)



- HTTP 4xx (400, 401, 403, 404)
- Validation errors
- Business rule violations
- Duplicate resource (409)
- Invalid request payload

Exponential Backoff Example (Base Delay = 1s, Max Retries = 4)



Notes

- Use jitter (randomness) in delays to avoid thundering herd.
- Respect Retry-After header for 429/503 responses.

GLOBAL HTTP ERROR STRATEGY

One interceptor, not fifty catchError() blocks -- centralize the parts of error handling that never change.



One Place for Common Cases

A single error interceptor handles 401 redirects and generic 500 messaging for every request in the app.



Components Handle the Rest

Field-level 400 validation errors still surface at the component that owns the form -- the interceptor doesn't own that.



Consistent User Messaging

A shared toast/snackbar service gives every unhandled error the same look and tone, app-wide.

KEY TAKEAWAY The 401-redirect-to-login behavior should exist in exactly one place in the codebase -- an interceptor, not copy-pasted into every component's error callback.

Global HTTP Error Strategy



A centralized approach to handle HTTP errors consistently across the entire Angular application.



WHY IT MATTERS

- Ensures consistent error handling and user experience
- Avoids duplicated error handling logic in every component
- Centralizes logging, user notifications, and fallback behavior
- Improves maintainability and debuggability

HOW IT WORKS



HttpClient
Request



HttpInterceptor
(Global Error
Handler)



HTTP Error
Response



User Notification
+ Logging
+ Fallback

IMPLEMENTATION EXAMPLE

```
@Injectable()
export class GlobalHttpErrorInterceptor implements HttpInterceptor {
  constructor(
    private errorHandler: ErrorHandler,
    private router: Router,
    private notification: NotificationService,
    private logger: LoggerService
  ) {}

  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    return next.handle(req).pipe(
      catchError((error: HttpResponse) => {
        this.logger.error('HTTP Error:', error);
        this.handleError(error);
        return throwError(() => new Error('Something went wrong'));
      })
    );
  }

  private handleError(error: HttpResponse): void {
    if (error.status === 401) {
      this.notification.show('Your session has expired. Please login again.');
```

ERROR HANDLING FLOW

- 1 HTTP Request** A request is made from any component using HttpClient.
- 2 Interceptor Catches Error** Global HttpInterceptor catches error responses.
- 3 Log the Error** Error details are logged for debugging and monitoring.
- 4 User Notification** A user-friendly message is shown using a notification service.
- 5 Fallback / Redirect** Redirect to login, error page, or perform any fallback action.
- 6 Return Observable Error** Propagate error to the caller if further handling is needed.

BEST PRACTICES



Centralize Error Handling
Handle all HTTP errors in one place (Interceptor).



User-Friendly Messages
Show clear, actionable messages to users.



Log for Developers
Log full error details for debugging and monitoring.



Handle Auth Errors
Redirect to login on 401, show access denied on 403.



Avoid Swallowing Errors
Propagate or rethrow errors when appropriate.



Monitor in Production
Integrate with monitoring tools (e.g., Sentry, ELK).



KEY TAKEAWAY: A global HTTP error strategy ensures consistent user experience, reduces duplicated code, and improves observability across the entire application.

HTTP INTERCEPTORS OVERVIEW

A functional interceptor runs on every request/response passing through HttpClient -- one function, app-wide reach.



HttpInterceptorFn

A plain function (req, next) => next(req) that can inspect, modify, or short-circuit any outgoing request.



Chained, in Order

withInterceptors([authInterceptor, errorInterceptor]) runs them in array order, each calling next() to continue the chain.



Requests AND Responses

An interceptor can modify the outgoing request, and also tap into the response (or error) flowing back through next(req).

KEY TAKEAWAY An interceptor is middleware for HttpClient -- it sees every request this app makes, which is exactly why it is the right place for auth headers and global error handling.

HTTP Interceptors Overview



HTTP Interceptors in Angular provide a powerful way to intercept and modify HTTP requests and responses globally across the application.



WHAT IS IT?

An HTTP interceptor is a service that implements the `HttpInterceptor` interface. It sits in the middle of the request-response pipeline and can:

- Modify outgoing requests
- Modify incoming responses
- Handle errors
- Perform actions before/after requests



WHY USE INTERCEPTORS?

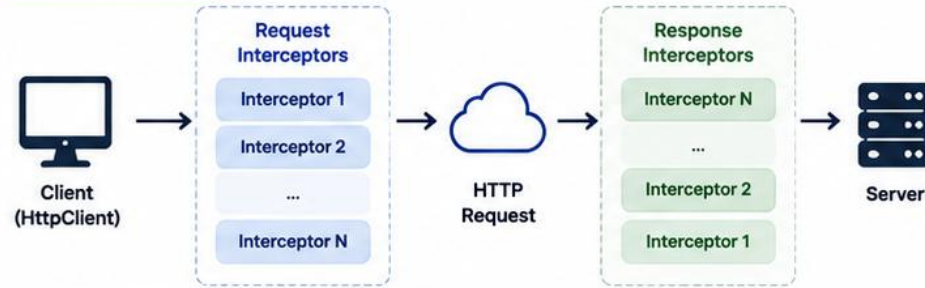
- ✓ Centralize cross-cutting concerns
- ✓ Reduce code duplication
- ✓ Keep components and services clean
- ✓ Improve maintainability and consistency



COMMON USE CASES

- 🔒 Add authentication tokens
- ⚠️ Handle global errors
- 📄 Log requests and responses
- 📄 Add or modify headers
- ⚙️ Show/hide loading indicators

HOW IT WORKS

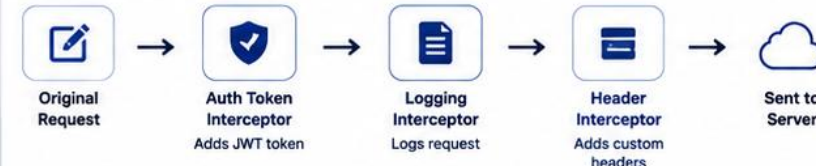


KEY POINTS

- ✓ Interceptors form a chain.
- ✓ Requests flow forward through the chain.
- ✓ Responses flow backward through the chain.
- ✓ Order matters. They execute in the order they are provided.
- ✓ Interceptors are global – apply to all HTTP calls.

EXAMPLE INTERCEPTOR CHAIN

OUTGOING REQUEST FLOW



INCOMING RESPONSE FLOW



BASIC INTERCEPTOR STRUCTURE

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>,
    next: HttpHandler): Observable<HttpEvent<any>> {
    const token = localStorage.getItem('token');
    const authReq = req.clone({
      setHeaders: {
        Authorization: `Bearer ${token}`
      }
    });
    return next.handle(authReq);
  }
}
```



Interceptors must be provided in the App Module using `HTTP_INTERCEPTORS` multi-provider.



KEY TAKEAWAY: HTTP Interceptors help you handle cross-cutting concerns like authentication, logging, and error handling in a centralized and consistent way, making your application cleaner, more secure, and easier to maintain.

AUTHENTICATION TOKEN INJECTION

One interceptor attaches the JWT to every protected request -- no service method does it manually.

STEP	WHAT HAPPENS
Read the token	authInterceptor reads the current JWT from a TokenService (backed by memory or storage).
Skip public routes	Requests to /api/auth/login are passed through unmodified -- no token exists yet.
Clone with header	req.clone({ setHeaders: { Authorization: `Bearer \${token}` } }) attaches it immutably.
Forward the request	return next(cloned) continues the chain with the modified request.
Handle 401 downstream	A follow-on error interceptor (or the same one) redirects to login on 401.

KEY TAKEAWAY If a protected endpoint returns 401 even though the user is logged in, the first thing to check is whether this interceptor is registered and running before the request leaves the browser.

TRY IT YOURSELF — EXERCISE 4: PLAN ERRORS AND INTERCEPTORS

Reinforces: HTTP error handling, retry considerations, the global error strategy, and token-injecting interceptors you just covered.

STEPS (notes/lab35-errors-interceptors.md)

Step	What To Do
1 — Status Map	Map 401, 403, 404, and 500 each to one specific UI behaviour.
2 — Retry Rule	Say which requests may be retried automatically and which must never be.
3 — Auth Interceptor	Say which requests get the token header and which are exempt.
4 — Correlation	Name the header that ties a UI failure to a backend log line.

Interceptor and error rules

```
401 -> clear token, go /login    403 -> inline 'not permitted'
retry: GET only (idempotent) -- never POST without an idempotency key
add: Authorization: Bearer ... and X-Correlation-Id: lab-request-001
```

TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-error-and-interceptors.md (workspace: examples/module-35-exercises).

Authentication Token Injection



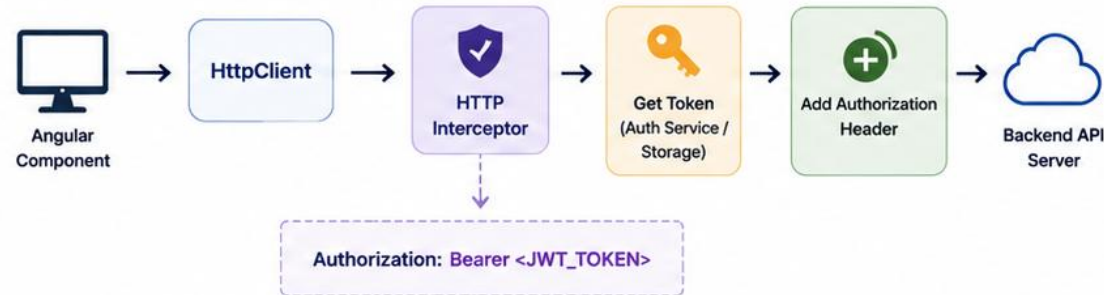
Automatically attach authentication tokens to every outgoing HTTP request using an Angular HTTP Interceptor.



WHY INJECT TOKENS?

- ✓ Ensures authenticated access to protected APIs
- ✓ Avoids adding Authorization header in every service call
- ✓ Centralized, reusable, and secure
- ✓ Easier token refresh and management

REQUEST FLOW WITH TOKEN INJECTION



BEST PRACTICES

- Store tokens securely**
Use secure storage (e.g., HttpOnly cookie, secure storage).
- Handle expired tokens**
Implement refresh logic in the interceptor or auth service.
- Skip public endpoints**
Don't attach token for login, register, or public APIs.
- Use environment configs**
Keep API base URLs and auth settings in environment files.



HOW IT WORKS

- 1 Request Intercepted**
Interceptor receives the outgoing request.
- 2 Get Token**
Retrieve the JWT token from storage or auth service.
- 3 Clone Request**
Clone the request and add the Authorization header.
- 4 Forward Request**
Send the modified request to the server.

EXAMPLE INTERCEPTOR (ANGULAR)

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    const token = localStorage.getItem('token');
    if (token) {
      const cloned = req.clone({
        setHeaders: {
          Authorization: `Bearer ${token}`
        }
      });
      return next.handle(cloned);
    }
    return next.handle(req);
  }
}
```

EXAMPLE OUTGOING REQUEST

Without Interceptor

```
GET /api/employees HTTP/1.1
Host: api.mycompany.com
Accept: application/json
```

With Interceptor

```
GET /api/employees HTTP/1.1
Host: api.mycompany.com
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

COMMON USE CASES

- JWT Authentication
- OAuth2 / OpenID Connect
- API Access Tokens
- Multi-tenant or Role-based APIs



KEY TAKEAWAY: Authentication token injection using an HTTP interceptor centralizes security, reduces boilerplate code, and ensures every protected API call is properly authenticated.

ENVIRONMENT-BASED API URLs

The Angular app must never hardcode `http://localhost:8080` -- every environment gets its own `apiUrl`.



Development

`environment.ts` points `apiUrl` at the local Spring Boot dev server, typically `http://localhost:8080/api`.



Test / Staging

A separate environment file points at a shared staging backend used for CI or manual QA before release.



Production

`environment.prod.ts` points `apiUrl` at the real production API domain, swapped in automatically at build time.

KEY TAKEAWAY Every `ApiService` builds its base URL from `environment.apiUrl`, never from a literal string -- that single line is what makes the same Angular build deployable to every environment.

Environment-Based API URLs



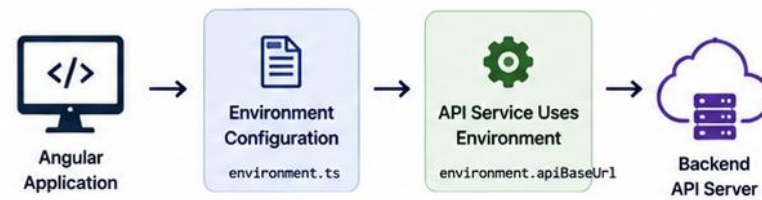
Use Angular environments to manage API base URLs across development, testing, staging, and production.



WHY USE ENVIRONMENT-BASED URLS?

- ✓ Different API endpoints per environment
- ✓ Avoid hardcoding URLs in code
- ✓ Safer deployments and easier configuration
- ✓ Simplifies switching between environments
- ✓ Supports local development and CI/CD pipelines

HOW IT WORKS



ENVIRONMENT FILES EXAMPLE

```
environment.ts (Development)
export const environment = {
  production: false,
  apiBaseUrl: 'http://localhost:8080/api'
};
```

```
environment.staging.ts (Staging)
export const environment = {
  production: false,
  apiBaseUrl: 'https://staging-api.myapp.com/api'
};
```

```
environment.prod.ts (Production)
export const environment = {
  production: true,
  apiBaseUrl: 'https://api.myapp.com/api'
};
```



TYPICAL PROJECT STRUCTURE

```
src/
├── environments/
│   ├── environment.ts
│   ├── environment.development.ts
│   ├── environment.staging.ts
│   └── environment.prod.ts
└── app/
```

Angular CLI automatically replaces environment.ts with the correct file during build.

USING IN CODE

```
// environment.ts
export const environment = {
  production: false,
  apiBaseUrl: 'http://localhost:8080/api'
};

// api.service.ts
import { environment } from '../environments/environment';

@Injectable({ providedIn: 'root' })
export class ApiService {
  private baseUrl = environment.apiBaseUrl;

  getEmployees() {
    return this.http.get<Employee[]>(`${this.baseUrl}/employees`);
  }
}
```

SWITCHING ENVIRONMENTS

	Description	Description
ng serve	Uses environment.ts (development)	
ng serve --configuration staging	Uses environment.staging.ts	
ng serve --configuration production	Uses environment.prod.ts	
ng build --configuration production	Builds with production environment	



BEST PRACTICES



Keep sensitive endpoints in environment files, not in code.



Use descriptive configuration names (staging, qa, prod).



Never commit real secrets or API keys.



Use Angular file replacements in CI/CD pipelines.



KEY TAKEAWAY

Environment-based API URLs make your application flexible, secure, and easy to deploy across multiple environments without code changes.

ANGULAR ENVIRONMENT CONFIGURATION

angular.json's fileReplacements swap environment.ts for the right file at build time -- automatically.

FILE / SETTING	ROLE
src/environments/environment.ts	Default file, imported by app code; used as-is for `ng serve`.
src/environments/environment.prod.ts	Production values; swapped in by `ng build` (production config).
fileReplacements (angular.json)	Maps environment.ts -> environment.prod.ts for the production build target.
import { environment } from '../environments/environment'	The only import path app code ever uses -- never imports the .prod file directly.
environment.production (boolean)	A flag some apps use to disable dev-only tooling (e.g. verbose logging) in prod.

KEY TAKEAWAY App code always imports the plain environment.ts path -- the build system, not the code, decides which actual file that import resolves to.



Angular Environment Configuration

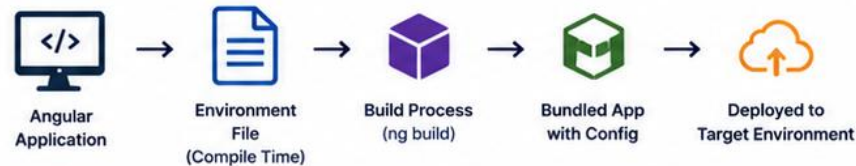
Angular environments help you manage configuration values like API base URLs, feature flags, and settings across different deployment targets.



WHY USE ENVIRONMENTS?

- ✓ Different configuration per environment (development, staging, production)
- ✓ Avoid hardcoding values in code
- ✓ Safe and consistent deployments
- ✓ Simplifies CI/CD and release management
- ✓ Supports feature flags and runtime settings

HOW IT WORKS



The correct environment file is selected during build time using the file replacement mechanism defined in angular.json.



TYPICAL PROJECT STRUCTURE



Angular CLI replaces environment.ts with the environment file matching the build configuration.

USING ENVIRONMENT IN CODE

```
// api.service.ts
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { environment } from '../environments/environment';

@Injectable({ providedIn: 'root' })
export class ApiService {
  private baseUrl = environment.apiUrl;

  constructor(private http: HttpClient) {}

  getEmployees() {
    return this.http.get(`${this.baseUrl}/employees`);
  }
}
```

SNIPPET FROM angular.json

```
{
  "configurations": {
    "production": {
      "fileReplacements": [
        {
          "replace": "src/environments/environment.ts",
          "with": "src/environments/environment.prod.ts"
        }
      ]
    }
  }
}
```

ENVIRONMENT FILE EXAMPLES

```
environment.ts (Default)
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8080/api',
  featureLogin: true,
  enableDebug: true
};
```

```
environment.staging.ts
export const environment = {
  production: false,
  apiUrl: 'https://staging-api.myapp.com/api',
  featureLogin: true,
  enableDebug: false
};
```

```
environment.prod.ts
export const environment = {
  production: true,
  apiUrl: 'https://api.myapp.com/api',
  featureLogin: false,
  enableDebug: false
};
```

BEST PRACTICES

- Never commit real secrets**
Use secure storage or environment variables for sensitive data.
- Use descriptive environment names**
E.g., development, staging, qa, production.
- Keep environment files in sync**
Maintain the same structure across all environment files.
- Use feature flags**
Control features per environment safely.



KEY TAKEAWAY: Angular environment configuration allows you to manage settings per environment cleanly and safely, enabling flexible builds and reliable deployments.

CORS

The browser, not Angular or Spring Boot, enforces Cross-Origin Resource Sharing -- and it blocks by default.



Same-Origin Policy

By default, a browser blocks JavaScript on one origin (localhost:4200) from reading responses from another (localhost:8080).



Server Opts In

CORS is the backend explicitly telling the browser 'this other origin is allowed to call me' via response headers.



Angular Can't Fix It Alone

No amount of Angular-side code can bypass a CORS block -- the fix always lives on the Spring Boot side.

KEY TAKEAWAY A CORS error in the browser console means the request reached the server and got a response -- the browser is the one refusing to hand that response to JavaScript.

CORS



Cross-Origin Resource Sharing (CORS) is a browser security mechanism that controls how web applications in one origin can request resources from another origin.



WHY CORS?

- ✓ Browsers block cross-origin HTTP requests (different domain, port, or protocol) by default.
- ✓ CORS allows servers to explicitly allow specific origins to access resources.
- ✓ Essential for Angular (frontend) calling Spring Boot (backend) APIs.



SAME-ORIGIN POLICY (BY DEFAULT)



KEY POINTS



CORS is enforced by browsers, not by servers.



Servers must explicitly allow trusted origins using CORS headers.

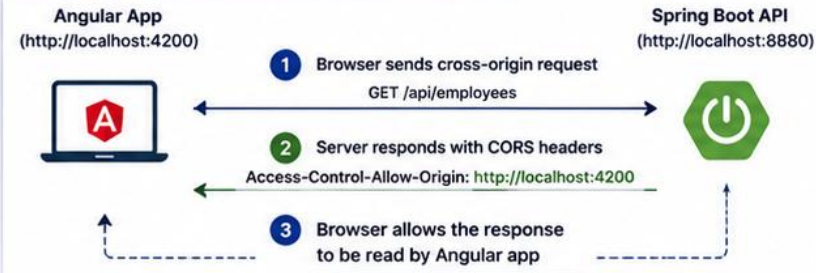


Required for local development and cross-domain apps.



Do not use * in production when credentials are involved.

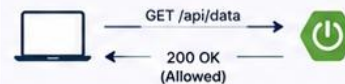
HOW CORS WORKS (HIGH LEVEL FLOW)



SIMPLE vs PRE-FLIGHTED REQUESTS

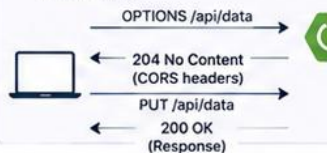
Simple Request

- Uses only safe methods: GET, POST, HEAD
- Headers: Accept, Content-Type (with certain values), etc.
- No pre-flight request is sent.



Pre-flight Request

- Uses methods like PUT, DELETE, PATCH or custom headers.
- Browser sends an OPTIONS pre-flight request first.



COMMON CORS HEADERS

Header	Purpose
<code>Access-Control-Allow-Origin</code>	Specifies which origin is allowed.
<code>Access-Control-Allow-Methods</code>	Specifies allowed HTTP methods.
<code>Access-Control-Allow-Headers</code>	Specifies allowed request headers.
<code>Access-Control-Allow-Credentials</code>	Indicates if credentials (cookies, auth headers) are allowed.
<code>Access-Control-Max-Age</code>	Time (in seconds) to cache pre-flight response.

EXAMPLE SCENARIO



BEST PRACTICES

- ✓ Allow only trusted origins (avoid * in production).
- ✓ Allow only necessary methods and headers.
- ✓ Use `Access-Control-Allow-Credentials: true` only when needed.
- ✓ Document and review CORS configuration regularly.



KEY TAKEAWAY: CORS enables secure and controlled cross-origin communication between your Angular frontend and Spring Boot backend by allowing the server to specify who can access its resources.

SPRING BOOT CORS CONFIGURATION

A CorsConfigurationSource bean is the production-grade fix -- @CrossOrigin is fine for a single controller.

APPROACH	WHEN TO USE IT
@CrossOrigin(origins = "http://localhost:4200")	Quick, controller-level fix for one endpoint or controller during early development.
WebMvcConfigurer.addCorsMappings()	App-wide CORS rule applied consistently across every controller.
CorsConfigurationSource bean	The Spring Security-integrated approach -- required once a SecurityFilterChain is in place.
allowedOrigins vs allowedOriginPatterns	Use exact origins in production; patterns only for flexible dev/preview URLs.
allowCredentials(true)	Required if Angular sends cookies or Authorization headers -- must pair with a specific origin, never a wildcard.

KEY TAKEAWAY Once Spring Security is in the picture, CORS must be configured through a CorsConfigurationSource bean wired into the SecurityFilterChain -- @CrossOrigin alone is not enough.

TRY IT YOURSELF — EXERCISE 5: CONFIGURE ENVIRONMENTS AND CORS

Reinforces: environment-based API URLs, Angular environment configuration, CORS, and the Spring Boot CORS setup you just covered.

STEPS (notes/lab35-environments-cors.md)

Step	What To Do
1 — Environment Files	Name the environment files and the property holding the API base URL.
2 — No Hard-Coding	Say why the base URL never appears inline in a service.
3 — CORS Cause	Explain in one sentence why the browser blocks it but curl does not.
4 — Boot Config	Name the origin, methods, and headers Spring Boot must allow.

Environment and CORS

```
environment.ts apiUrl: 'http://localhost:8080'    (dev)
environment.prod.ts apiUrl: 'https://crm.example.com'
Boot allows origin http://localhost:4200, methods GET..OPTIONS, header X-Correlation-Id
```

TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-environments-and-cors.md (workspace: examples/module-35-exercises).

Spring Boot CORS Configuration

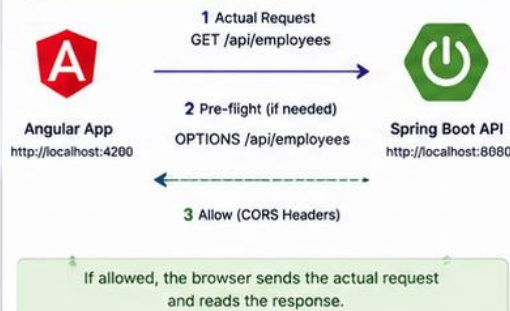
Configure Cross-Origin Resource Sharing (CORS) in Spring Boot to allow secure access from your Angular frontend or other clients.



WHY CORS?

- ✓ Browsers block cross-origin requests by default.
- ✓ CORS allows your API to explicitly permit specific origins.
- ✓ Required for Angular apps calling Spring Boot APIs from a different origin (domain/port).
- ✓ Helps maintain security while enabling controlled access.

CORS REQUEST FLOW



BEST PRACTICES



Allow only trusted origins.



Use specific methods and headers.



Enable allowCredentials(true) only when needed.



Set a reasonable maxAge to reduce pre-flight calls.



Test CORS with both pre-flight and actual requests.



KEY TAKEAWAY: Proper CORS configuration in Spring Boot ensures secure and reliable communication between your frontend and backend applications while protecting your API from unauthorized access.

1. GLOBAL CORS CONFIGURATION (Recommended)

Using WebMvcConfigurer

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
            .allowedOrigins("http://localhost:4200", "https://myapp.com")
            .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS")
            .allowedHeaders("*")
            .allowCredentials(true)
            .maxAge(3600);
    }
}
```

Notes

- /** : Apply to all endpoints
- allowedOrigins : List of allowed origins
- allowedMethods : HTTP methods permitted
- allowedHeaders : Allowed request headers
- allowCredentials : Allow cookies/auth headers
- maxAge : Cache duration (in seconds) for pre-flight response

2. APPLICATION.PROPERTIES (Basic CORS)

spring.mvc.cors mappings

```
spring.mvc.cors.mapping."/**".allowed-origins=http://localhost:4200,https://myapp.com
spring.mvc.cors.mapping."/**".allowed-methods=GET,POST,PUT,PATCH,DELETE,OPTIONS
spring.mvc.cors.mapping."/**".allowed-headers=*
spring.mvc.cors.mapping."/**".allow-credentials=true
spring.mvc.cors.mapping."/**".max-age=3600
```

ALLOWING ALL ORIGINS (Not for Production)

```
registry.addMapping("/**")
    .allowedOrigins("*")
    .allowedMethods("*")
    .allowedHeaders("*");
```



Using "*" with allowCredentials(true) is not allowed by browsers.

PER-CONTROLLER / METHOD CORS

Using @CrossOrigin

```
@RestController
@RequestMapping("/api/employees")
@CrossOrigin(origins = "http://localhost:4200")
public class EmployeeController {

    @GetMapping
    public List<Employee> getAll() {
        return service.getAll();
    }
}
```

COMMON HEADERS RETURNED BY CORS

Header	Purpose
Access-Control-Allow-Origin	Specifies allowed origin
Access-Control-Allow-Methods	Specifies allowed HTTP methods
Access-Control-Allow-Headers	Specifies allowed request headers
Access-Control-Allow-Credentials	Indicates if credentials are allowed
Access-Control-Max-Age	Cache time (seconds) for pre-flight

API SERVICE DESIGN BEST PRACTICES

A handful of consistent habits keep a growing set of Angular services maintainable.



Name by Resource

CustomerApiService, OrderApiService -- one focused service per resource, never one giant ApiService.



Always Type the Return

Every method declares Observable<T> explicitly -- never leave it inferred as Observable<any>.



No Business Logic Here

Services fetch and shape data; validation and business rules live in the component or a dedicated domain layer.

KEY TAKEAWAY An API service's job is narrow on purpose: build the URL, make the typed call, shape the response -- nothing else.

API Service Design Best Practices



Well-designed Angular API services improve maintainability, performance, security, and developer productivity.



PRINCIPLES



Single Responsibility

Each service handles communication with a specific API or domain.



Abstraction

Components should not know HTTP details. Services hide implementation.



Reusability

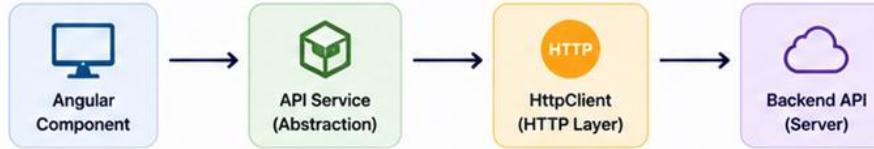
Share services across components and features.



Testability

Services should be easy to mock and test in isolation.

RECOMMENDED SERVICE ARCHITECTURE



BENEFITS

- ✓ Clean separation of concerns
- ✓ Easy to test and mock
- ✓ Centralized error handling & logging
- ✓ Consistent request / response handling
- ✓ Easier maintenance and scalability
- ✓ Supports interceptors & caching

EXAMPLE: EMPLOYEE API SERVICE (ANGULAR)

```
@Injectable({ providedIn: 'root' })
export class EmployeeService {
  private baseUrl = environment.apiUrl + '/employees';
  constructor(private http: HttpClient) {}

  getAll(): Observable<Employee[]> {
    return this.http.get<Employee[]>(this.baseUrl);
  }
  getById(id: number): Observable<Employee> {
    return this.http.get<Employee>(`${this.baseUrl}/${id}`);
  }
  create(employee: Employee): Observable<Employee> {
    return this.http.post<Employee>(this.baseUrl, employee);
  }
  update(id: number, employee: Employee): Observable<Employee> {
    return this.http.put<Employee>(`${this.baseUrl}/${id}`, employee);
  }
  delete(id: number): Observable<void> {
    return this.http.delete<void>(`${this.baseUrl}/${id}`);
  }
}
```

KEY POINTS

- 🔗 Base URL comes from environment
- 📄 Strongly typed requests & responses
- ➕ Returns Observables (no subscribe in service)
- ⚠️ Errors handled in interceptor or service layer

BEST PRACTICES CHECKLIST

- ✓ Use a dedicated service per domain/resource
- ✓ Keep services thin and focused on data access
- ✓ Return Observables and let components subscribe
- ✓ Use typed interfaces for requests and responses
- ✓ Use environment files for API base URLs
- ✓ Handle errors consistently (interceptor or service)
- ✓ Use HttpParams for query parameters
- ✓ Use HttpHeaders for custom headers
- ✓ Enable caching where appropriate
- ✓ Write unit tests with HttpClientTestingModule

COMMON ANTI-PATTERNS

- ✗ Embedding API calls directly in components
- ✗ Creating multiple services for the same resource
- ✗ Hardcoding full API URLs
- ✗ Subscribing inside services
- ✗ Swallowing errors or returning undefined
- ✗ Returning any and losing type safety

DOs AND DON'Ts

- | | |
|--|--|
| ✓ Do use HttpClient (not fetch/XHR) | ✗ Don't call APIs directly from components |
| ✓ Do centralize API calls in services | ✗ Don't duplicate HTTP logic |
| ✓ Do return Observables | ✗ Don't subscribe inside services |
| ✓ Do handle errors in one place | ✗ Don't hardcode URLs |
| ✓ Do use typed models (interfaces) | ✗ Don't ignore error handling |
| ✓ Do use environment-based URLs | ✗ Don't return any or untyped responses |
| ✓ Do use interceptors for cross-cutting concerns | |



KEY TAKEAWAY: A well-designed API service layer promotes clean architecture, reusability, testability, and reliability across your Angular application. Keep it **simple**, **consistent**, and **maintainable**.

COMPONENT -> SERVICE -> API FLOW

Three layers, one direction of dependency -- component depends on service, service depends on HttpClient.



Component

Owns the template and presentation state (loading, error, data) -- calls the service, never HttpClient directly.



Service

Owns the URL, the typed call, and any response shaping -- calls HttpClient, never touches the DOM.



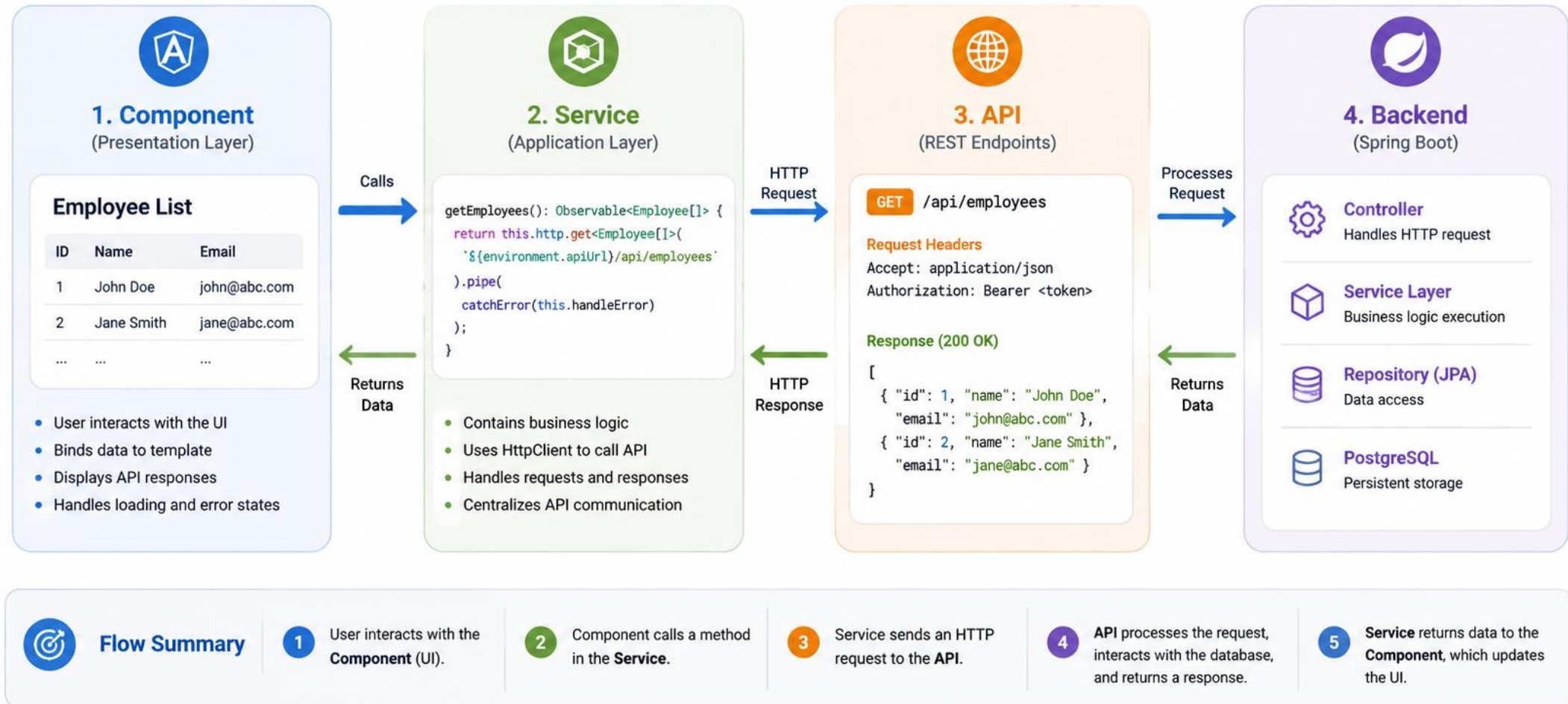
HttpClient

Owns the actual network transport, interceptors, and JSON (de)serialization.

KEY TAKEAWAY The dependency arrow only points one way: component -> service -> HttpClient -- a service never reaches back up into a component, and HttpClient never knows a component exists.

Component → Service → API Flow

The typical data flow in an Angular application communicating with a Spring Boot REST API.



CRUD WORKFLOW

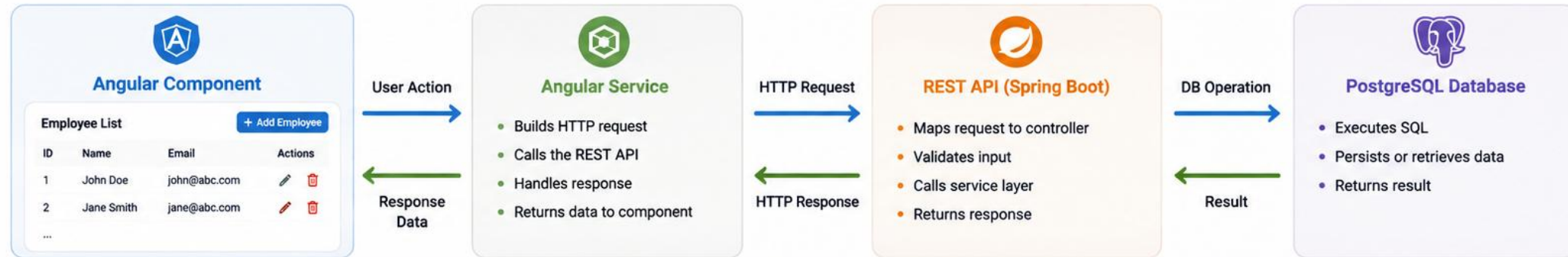
Create, Read, Update, Delete -- the same four operations, mapped consistently across every resource.

OPERATION	HTTP VERB	ANGULAR SERVICE METHOD
Create	POST /api/customers	createCustomer(dto): Observable<Customer>
Read (list)	GET /api/customers	getCustomers(params): Observable<Customer[]>
Read (one)	GET /api/customers/{id}	getCustomer(id): Observable<Customer>
Update	PUT or PATCH /api/customers/{id}	updateCustomer(id, dto): Observable<Customer>
Delete	DELETE /api/customers/{id}	deleteCustomer(id): Observable<void>

KEY TAKEAWAY Every resource in this course's Angular app should expose the same five-method CRUD shape -- consistency here is what makes a new resource's service quick to write and easy to review.

CRUD Workflow

The standard Create, Read, Update, Delete operations in a full-stack application.



Operation	HTTP Method	Endpoint Example	Description	Component Action	Service Action	DB Action	Result
CREATE	POST	/api/employees	Creates a new employee record.	Submit create form or click "Add"	Send POST request with employee data	INSERT new record into employees table	201 Created New employee returned
READ	GET	/api/employees /api/employees/{id}	Retrieves employee list or details.	Load list or view details	Send GET request (with filters/pagination)	SELECT data from employees table	200 OK Employee data returned
UPDATE	PUT	/api/employees/{id}	Updates an existing employee record.	Edit form and click "Save"	Send PUT request with updated data	UPDATE existing record in employees table	200 OK Updated employee returned
DELETE	DELETE	/api/employees/{id}	Deletes an employee record.	Click "Delete" icon	Send DELETE request with employee ID	DELETE record from employees table	204 No Content Employee deleted

Key Benefits



Standardized operations across all resources



Predictable and consistent API behavior



Easier testing and automation



Scalable and maintainable architecture



Clear security and validation boundaries



Best Practice: Follow REST conventions, use proper HTTP methods, meaningful endpoints, and return appropriate status codes.

FORM SUBMISSION TO REST API

A Reactive Form's value becomes the request body -- but only after it passes client-side validation.



Reactive Form Value

`this.customerForm.value` gives the raw form data -- shape it into the request DTO before sending.



Validate Before Sending

`form.valid` gates the submit handler -- an invalid form never reaches the HTTP call at all.



Disable While Submitting

The submit button disables during the pending request to prevent a duplicate, accidental double-POST.

KEY TAKEAWAY Client-side validation is a UX convenience, not a security boundary -- Spring Boot's `@Valid` still runs and still matters, because client-side checks can always be bypassed.

Form Submission to REST API

User fills a form in Angular, submits it to a Spring Boot REST API, and receives a response.

1 User Fills the Form

User enters data and clicks Submit.

Add Employee

Name

John Doe

Email

john.doe@abc.com

Department

Engineering

Salary

75000

Submit

2 Angular Sends Request

HttpClient posts form data as JSON to the API.

```
TS
onSubmit() {
  const payload: EmployeeDto = {
    name: this.form.value.name,
    email: this.form.value.email,
    department: this.form.value.department,
    salary: this.form.value.salary
  };

  this.employeeService.createEmployee(payload)
    .subscribe({
      next: (res) => this.onSuccess(res),
      error: (err) => this.onError(err)
    });
}
```

3 REST API Processes Request

Spring Boot validates, saves data, and returns a response.

Spring Boot

POST /api/employees

Request Body (JSON)

```
{
  "name": "John Doe",
  "email": "john.doe@abc.com",
  "department": "Engineering",
  "salary": 75000
}
```

Response (201 Created)

```
{
  "id": 101,
  "name": "John Doe",
  "email": "john.doe@abc.com",
  "department": "Engineering",
  "salary": 75000,
  "createdAt": "2024-05-18T10:30:00Z"
}
```

4 Angular Handles Response

Component processes response and updates the UI.

```
TS
onSuccess(res: Employee) {
  // Show success message
  this.toastr.success('Employee created successfully!');
  // Reset form
  this.form.reset();
  // Reload list
  this.loadEmployees();
}

onError(err: any) {
  this.toastr.error(err.error?.message || 'Failed to create employee');
}
```

5 UI Updated

New employee appears in the list.

Employee List		
ID	Name	Email
101	John Doe	john.doe@abc.com
100	Jane Smith	jane.smith@abc.com
99	Mark Lee	mark.lee@abc.com
...	...	...

End-to-End Flow



User completes and submits the form.



Angular Form captures data and sends HTTP POST.



Spring Boot API validates and persists data to the database.



API returns created resource with status 201 Created.



Angular updates the UI and displays the new employee.

Best Practices



Use Reactive Forms for validation and better control.



Send only necessary fields in the payload (DTO).



Validate on both client and server side.



Handle errors gracefully and show meaningful messages.



Reset form and refresh data after successful submission.



Use HTTPS and proper authentication for secure APIs.

DISPLAYING BACKEND VALIDATION ERRORS

A 400 response carries structured, field-level errors -- map them back onto the exact form controls that failed.

PROBLEM DETAILS FIELD	ANGULAR HANDLING
status: 400	Distinguish from other error codes in the error callback -- this is a validation failure, not a server fault.
errors: [{ field, message }]	Iterate the array and call form.get(field)?.setErrors({ server: message }) per entry.
title / detail	Show as a general form-level banner if no field-specific errors are present.
Re-enabling the form	isSubmitting resets to false so the user can correct fields and resubmit.
Clearing stale errors	Server-set errors should clear on the next valueChanges event for that control.

KEY TAKEAWAY Field-level backend errors belong next to the field that failed, not in one generic banner at the top of the form -- that's the difference between a usable form and a frustrating one.

Displaying Backend Validation Errors

Show meaningful validation messages returned by the backend in the Angular UI.



REQUEST (POST /api/employees)

```
{  "name": "",  "email": "invalid-email",  "department": "",  "salary": -5000}
```

HTTP POST

RESPONSE (400 Bad Request)

```
{  "timestamp": "2024-05-18T10:30:00Z",  "status": 400,  "error": "Bad Request",  "message": "Validation failed",  "path": "/api/employees",  "errors": [    { "field": "name", "message": "Name is required" },    { "field": "email", "message": "Email must be valid" },    { "field": "department", "message": "Department is required" },    { "field": "salary", "message": "Salary must be greater than 0" }  ]}
```

HTTP 400

ANGULAR ERROR HANDLING (Example)

```
this.employeeService.createEmployee(payload)  .subscribe({    next: (res) => this.onSuccess(res),    error: (err: HttpResponse) => {      if (err.status === 400 && err.error?.errors) {        this.fieldErrors = this.mapErrors(err.error.errors);      } else {        this.generalError = 'Unable to save employee.';      }    }  });mapErrors(errors: any[]): { [key: string]: string } {  const map: any = {};  errors.forEach(e => map[e.field] = e.message);  return map;}
```

Why This Matters

- ✓ Provides clear, field-level feedback to users
- ✓ Reduces confusion and support tickets
- ✓ Ensures consistent validation between frontend and backend
- ✓ Improves user experience and data quality

Common Backend Validation → UI Mapping			
Backend Status	Error Type	Example	UI Handling
400	Field Errors	Name is required	Show next to the field
400	Constraint Violation	Email must be valid	Show field-level message
409	Duplicate	Email already exists	Show inline or toast
500	Server Error	Internal server error	Show generic error message

Add Employee

Name *

Enter name

Name is required

Email *

invalid-email

Email must be valid

Department *

Select department

Department is required

Salary *

-5000

Salary must be greater than 0

Save Employee

!

Please fix the errors above and try again.

Best Practices

✓

Use consistent error response format across APIs

🏷️

Return clear and user-friendly error messages

</>

Map errors to specific form fields wherever possible

🔄

Keep frontend and backend validation rules aligned

ⓘ

Show a global error for non-field or unexpected errors

PAGINATION INTEGRATION

Spring Data's Pageable maps to page/size query params; the response envelope carries the metadata a UI needs.



page & size Params

getCustomers(page: number, size: number)
builds HttpParams for ?page=0&size=20,
matching Spring Data's Pageable.



Page<T> Envelope

The response includes content,
totalElements, totalPages -- not just a bare
array -- so the UI can render page controls.



Zero-Based Pages

Spring Data pages are zero-indexed -- page
1 in the UI usually means page: 0 in the
request.

KEY TAKEAWAY Don't discard the pagination metadata -- totalElements and totalPages from the response are what let the UI render 'Page 3 of 12,' not just the current page's rows.

Pagination Integration

Load and display large data sets in pages for better performance and user experience.



Employees List

ID	Name	Email	Department
101	John Doe	john.doe@abc.com	Engineering
102	Jane Smith	jane.smith@abc.com	HR
103	Mark Lee	mark.lee@abc.com	Finance
104	Sarah Johnson	sarah.j@abc.com	Engineering
105	David Brown	david.b@abc.com	Operations

Showing 1 to 5 of 42 employees

« < 1 2 3 4 5 > » 5 / page ▾

Angular Service (HttpClient) TS

```
getEmployees(page: number, size: number, sort: string):  
Observable<Page<Employee>> {  
  const params = new HttpParams()  
    .set('page', page.toString()) // 0-based  
    .set('size', size.toString()) // e.g. name,asc  
    .set('sort', sort);  
  
  return this.http.get<Page<Employee>>('/api/employees', { params });  
}
```

TypeScript Interface TS

```
export interface Page<T> {  
  content: T[];  
  page: {  
    number: number; // current page (0-based)  
    size: number; // page size  
    totalElements: number; // total items  
    totalPages: number; // total pages  
  };  
  sort: {  
    sorted: boolean; unsorted: boolean; empty: boolean;  
    first: boolean;  
    last: boolean;  
    numberOfElements: number;  
    empty: boolean;  
  };  
}
```

Component Usage TS

```
currentPage = 0;  
pageSize = 5;  
sort = 'name,asc';  
employees: Employee[] = [];  
totalPages = 0;  
  
load(page: number = 0) {  
  this.employeeService.getEmployees(page, this.pageSize, this.sort)  
    .subscribe(res => {  
      this.employees = res.content;  
      this.totalPages = res.page.totalPages;  
      this.currentPage = res.page.number;  
    });  
}  
  
next() { if (this.isEnabled()) this.load(this.currentPage + 1); }  
prev() { if (this.currentPage > 0) this.load(this.currentPage - 1); }  
isEnabled() { return this.currentPage < this.totalPages - 1; }
```

Backend API (Spring Boot)

GET /api/employees?page=0&size=5&sort=name,asc

Response (200 OK)

```
{  
  "content": [  
    { "id": 101, "name": "John Doe", "email": "john.doe@abc.com" },  
    { "id": 102, "name": "Jane Smith", "email": "jane.smith@abc.com" },  
    { "id": 103, "name": "Mark Lee", "email": "mark.lee@abc.com" },  
    { "id": 104, "name": "Sarah Johnson", "email": "sarah.j@abc.com" },  
    { "id": 105, "name": "David Brown", "email": "david.b@abc.com" }  
  ],  
  "page": { "number": 0, "size": 5, "totalElements": 42, "totalPages": 9 },  
  "sort": { "sorted": true, "unsorted": false, "empty": false },  
  "first": true, "last": false, "numberOfElements": 5,  
  "empty": false  
}
```

Key Points

- ✓ Use **Pageable** in Spring Data JPA for pagination.
- ✓ Default page index is 0.
- ✓ Return **Page<T>** for metadata and content.
- ✓ Always include **sort** for consistent results.
- ✓ Keep page size reasonable (e.g., 5, 10, 20).



Best Practices



Use keyset pagination for very large datasets.



Validate and cap maximum page size.



Provide default sorting.



Consider caching for read-heavy lists.



Return only necessary fields (projection).

SORTING INTEGRATION

sort=lastName,asc is a single query parameter Spring Data parses directly into an order-by clause.



sort Parameter Format

sort=field,direction (e.g. lastName,asc) -- Spring Data's Pageable parses this automatically.



Multi-Field Sort

Repeating &sort=field,direction for multiple fields applies a compound sort, in the order given.



UI-Driven Sort State

A sortable column header toggles the current sort field/direction, rebuilding the request params on click.

KEY TAKEAWAY Sorting is entirely a query-parameter concern -- Angular never sorts the array client-side once the backend already applied a database-level ORDER BY.

Sorting Integration

Enable users to sort data by one or more fields in ascending or descending order.



UI: Sort Controls

Sort by

Order

Name

Ascending (A → Z)

ID	Name	Email	Department
101	Alice Johnson	alice@abc.com	Engineering
102	Bob Smith	bob@abc.com	HR
103	Charlie Brown	charlie@abc.com	Finance
104	David Lee	david@abc.com	Engineering
105	Emma Wilson	emma@abc.com	Operations

<< < 1 2 3 > >>

5 / page

Angular Service (HttpClient)

```
getEmployees(page: number, size: number, sortBy: string, direction: 'asc' | 'desc'): Observable<Page<Employee>> {
  const params = new HttpParams()
    .set('page', page.toString())
    .set('size', size.toString())
    .set('sort', `${sortBy},${direction}`); // e.g. name,asc

  return this.http.get<Page<Employee>>('/api/employees', { params });
}
```

Example Request

```
GET /api/employees?page=0&size=5&sort=department,desc
```

- sort format: field,direction
- direction: asc (default) or desc
- multiple fields: sort=department,asc&sort=name,desc

Backend API (Spring Boot)

```
GET /api/employees?page=0&size=5&sort=name,asc

Controller
@GetMapping
public Page<Employee> getEmployees(
    @RequestParam int page,
    @RequestParam int size,
    @RequestParam(required = false) String sort) {
    Sort sortObj = Sort.by(Sort.Direction.ASC, "name");
    if (sort != null && !sort.isBlank()) {
        String[] parts = sort.split(","); // field,direction
        String field = parts[0];
        Sort.Direction dir = parts.length > 1 && parts[1].equalsIgnoreCase("desc")
            ? Sort.Direction.DESC : Sort.Direction.ASC;
        sortObj = Sort.by(dir, field);
    }
    Pageable pageable = PageRequest.of(page, size, sortObj);
    return employeeService.findAll(pageable);
}
```

Common Sort Options

Name (A → Z)	name, asc
Name (Z → A)	name, desc
Department (A → Z)	department, asc
Salary (High → Low)	salary, desc
Multiple Fields	department, asc&name, asc

Example Response (200 OK)

```
{
  "content": [
    { "id": 105, "name": "Emma Wilson", "department": "Operations", ... },
    { "id": 102, "name": "Bob Smith", "department": "HR", ... },
    { "id": 103, "name": "Charlie Brown", "department": "Finance", ... },
    { "id": 101, "name": "Alice Johnson", "department": "Engineering", ... },
    { "id": 104, "name": "David Lee", "department": "Engineering", ... }
  ],
  "page": { "number": 0, "size": 5, "totalElements": 42, "totalPages": 9 },
  "sort": { "sorted": true, "unsorted": false, "empty": false }
}
```

- Key Points**
- ✓ Use Spring Data Sort with Pageable.
 - ✓ Default to a safe sort (e.g., name,asc).
 - ✓ Whitelist allowed sort fields to prevent misuse.
 - ✓ Return sort metadata in the response.
 - ✓ Ensure indexes exist on frequently sorted fields.

Best Practices

Validate and whitelist sortable fields.

Provide a default sort for consistency.

Use database indexes for better performance.

Keep sorting parameters simple and predictable.

Consider combined indexes for multi-field sorting.

FILTERING INTEGRATION

Each active filter becomes one more optional query parameter -- only set when the user actually chose a value.



status / search Params

?status=ACTIVE&search=amina -- each filter maps to a Spring Data Specification or a derived repository query.



Debounce Free-Text Search

A search box should debounce (e.g. 300ms) before firing a request -- not one HTTP call per keystroke.



Combine with Page Reset

Changing any filter should reset the request back to page 0 -- staying on page 5 of a now-different result set is confusing.

KEY TAKEAWAY A search box wired directly to (input) without debouncing turns every keystroke into a wasted HTTP request -- combineLatest and debounceTime fix that cleanly.

Filtering Integration

Allow users to filter data using query parameters and display only the relevant results.



UI: Employee Filters

Department

Status

Min Salary

Max Salary

Search Name

Active Filters

Department: Engineering X

Status: ACTIVE X

Min Salary: 60000 X

Angular Service (HttpClient)

```
getEmployees(filters: EmployeeFilter): Observable<Page<Employee>> {
  const params = new HttpParams({
    fromObject: {
      department: filters.department || '',
      status: filters.status || '',
      minSalary: filters.minSalary?.toString() || '',
      maxSalary: filters.maxSalary?.toString() || '',
      name: filters.name || '',
      page: filters.page?.toString() || '0',
      size: filters.size?.toString() || '10'
    }
  });
  return this.http.get<Page<Employee>>(`/api/employees`, { params });
}
```

Example Request

```
GET /api/employees?department=Engineering&status=ACTIVE&minSalary=60000&maxSalary=120000&name=John&page=8&size=10&sort=salary,desc
```

Query Parameters

• department	→ Filter by department	• maxSalary	→ Maximum salary
• status	→ Filter by status	• page, size	→ Search by name (contains)
• minSalary	→ Minimum salary	• sort	→ Sorting

Example Response (200 OK)

```
{
  "content": [
    { "id": 101, "name": "John Doe", "department": "Engineering", "status": "ACTIVE", "salary": 95000 },
    { "id": 115, "name": "John Smith", "department": "Engineering", "status": "ACTIVE", "salary": 80000 }
  ],
  "page": { "number": 0, "size": 10, "totalElements": 17, "totalPages": 2 },
  "sort": { "sorted": true, "empty": false }
}
```

Backend API (Spring Boot)

```
GET /api/employees?department=Engineering&status=ACTIVE&minSalary=60000&name=John

@GetMapping
public Page<Employee> getEmployees(EmployeeFilter filter,
                                   Pageable pageable) {
    return employeeService.getEmployees(filter, pageable);
}

public Page<Employee> getEmployees(EmployeeFilter filter, Pageable pageable) {
    Specification<Employee> spec = Specification.where(null);

    if (StringUtils.hasText(filter.getDepartment())) {
        spec = spec.and(EmployeeSpecs.departmentEquals(filter.getDepartment()));
    }
    if (StringUtils.hasText(filter.getStatus())) {
        spec = spec.and(EmployeeSpecs.statusEquals(filter.getStatus()));
    }
    if (filter.getMinSalary() != null) {
        spec = spec.and(EmployeeSpecs.salaryGreaterThanOrEqual(filter.getMinSalary()));
    }
    if (filter.getMaxSalary() != null) {
        spec = spec.and(EmployeeSpecs.salaryLessThanOrEqual(filter.getMaxSalary()));
    }
    if (StringUtils.hasText(filter.getName())) {
        spec = spec.and(EmployeeSpecs.nameContains(filter.getName()));
    }
    return employeeRepository.findAll(spec, pageable);
}
```

Database Query (Generated)

```
SELECT e.id, e.name, e.department, e.status, e.salary
FROM employee e
WHERE (:department IS NULL OR e.department = :department)
AND (:status IS NULL OR e.status = :status)
AND (:minSalary IS NULL OR e.salary >= :minSalary)
AND (:maxSalary IS NULL OR e.salary <= :maxSalary)
AND (:name IS NULL OR LOWER(e.name) LIKE LOWER(CONCAT('%', :name, '%')))
ORDER BY e.salary DESC
LIMIT :size OFFSET :offset;
```

Key Points

- ✓ Use query parameters for filtering.
- ✓ Keep filters optional and composable.
- ✓ Use Specification (Spring Data JPA) for dynamic filters.
- ✓ Validate and sanitize all filter inputs.
- ✓ Index frequently filtered columns.
- ✓ Combine filtering with pagination and sorting.

Best Practices

Use meaningful query parameter names.

Support partial filters (all optional).

Document supported filters in OpenAPI.

Ensure proper indexes on filtered columns.

Return metadata with filtered results.

Guard against expensive queries (large ranges).

DEBUGGING NETWORK REQUESTS

When a call 'doesn't work,' the network tab tells you whether it was sent, and what came back.



Was It Sent At All?

No entry in the Network tab usually means the Observable was never subscribed to -- the classic cold-Observable trap.



What Was Actually Sent

Inspect the request payload and headers -- confirm the body and Authorization header look exactly as expected.



What Came Back

Check the response status, body, and headers -- a CORS block looks different from a 401, which looks different from a 500.

KEY TAKEAWAY Before touching any code, answer one question first: did a request even leave the browser? Everything else follows from that answer.

Debugging Network Requests

Use browser DevTools to inspect, analyze, and troubleshoot API calls between Angular and Spring Boot.



Common Steps

- Open DevTools (F12)**
Go to the Network tab.
- Reproduce the Action**
Click the button or perform the action that triggers the request.
- Find the Request**
Locate the API call in the Network request list.
- Inspect Details**
Check Headers, Payload, Response, Status, and Timing.
- Identify Issues**
Use the insights to fix request, response, or performance issues.

What to Check

- Request URL and HTTP Method
- Query Parameters and Path Variables
- Request Headers (Content-Type, Authorization, etc.)
- Request Payload / Request Body
- Response Status Code
- Response Headers
- Response Body
- Timing and Size

Elements Console Sources **Network** Performance Memory Application Security >>

Filter ☐ Invert ☐ Hide data URLs **All** Fetch/XHR JS CSS Img Media Font Doc WS Wasm Other

100 ms 200 ms 300 ms 400 ms 500 ms 600 ms 700 ms 800 ms 900 ms 1000 ms

Name	Headers	Payload	Preview	Response	Timing
☒ employees?page=0&size=10&sort=name,asc	General Request URL: http://localhost:8080/api/employees?page=0&size=10&sort=name,asc Request Method: GET Status Code: 200 OK Remote Address: [::1]:8080 Referrer Policy: strict-origin-when-cross-origin				
☐ employees/101					
☐ employees					
☐ departments					
☐ auth/login					
☐ refresh-token					
☒ favicon.ico	Response Headers Content-Type: application/json Transfer-Encoding: chunked Date: Tue, 20 May 2025 12:15:30 GMT				
	Request Headers Accept: application/json, text/plain, */* Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... Content-Type: application/json X-Requested-With: XMLHttpRequest				
	Query String Parameters page: 0 size: 10 sort: name,asc	view source	view decoded		

7 requests | 1.2 MB transferred | 1.8 MB resources

Request-Response Flow

Request
GET /api/employees?page=0&size=10&sort=name,asc
Headers
Accept: application/json
Authorization: Bearer ...

Response
HTTP 200 OK
Headers
Content-Type: application/json
Body (JSON)
{
 "content": [
 { "id": 101, "name": "John Doe" },
 { "id": 102, "name": "Jane Smith" }
],
 "totalElements": 42,
 "totalPages": 5,
 "size": 10,
 "number": 0
}

Common Issues and Fixes

	404 Not Found	Check the URL, path variables, and endpoint mapping.
	400 Bad Request	Validate query params, request body, and data types.
	401/403 Unauthorized	Verify authentication token and permissions.
	500 Server Error	Check server logs and request payload.
	Slow Requests	Check payload size, database queries, and network timing.

Pro Tips

Use Filters
Filter by Fetch/XHR to see API calls only.

Preview Tab
Quickly view JSON responses.

Timing Tab
Analyze request duration and breakdown.

Copy as cURL
Reproduce requests in Postman/cURL.

Disable Cache
Ensure you're seeing fresh data.

BROWSER DEVELOPER TOOLS

Three panels do almost all the work: Network, Console, and Application.

PANEL	WHAT TO CHECK
Network tab	Every outgoing request, its status, timing, headers, and payload -- the primary triage tool.
Console tab	CORS errors, uncaught RxJS errors, and any console.error() logging from the app itself.
Application tab	Inspect stored tokens/state (e.g. sessionStorage) to confirm auth data is present and current.
Preserve log	Enable this in the Network tab before testing a redirect or page reload -- otherwise history is lost.
Throttling	Simulate a slow connection to verify loading states actually appear and don't flash instantly.

KEY TAKEAWAY Turn on 'Preserve log' before testing anything involving a redirect or reload -- otherwise the very request you need to inspect disappears the moment the page navigates.



MODULE 35

Frontend to API Integration

Browser Developer Tools

Inspect. Debug. Understand your frontend-backend integration.

Browser Developer Tools help you inspect UI, monitor network requests, analyze responses, and debug frontend behavior.



Inspect Network Requests

View all HTTP requests made by your application, including method, status code, headers, query params, and payload.



Analyze Responses

Preview JSON responses, check response time, status codes, and validate data returned by your API.



Debug Frontend Behavior

Use Console, Sources, and Breakpoints to identify JavaScript errors and debug application logic.

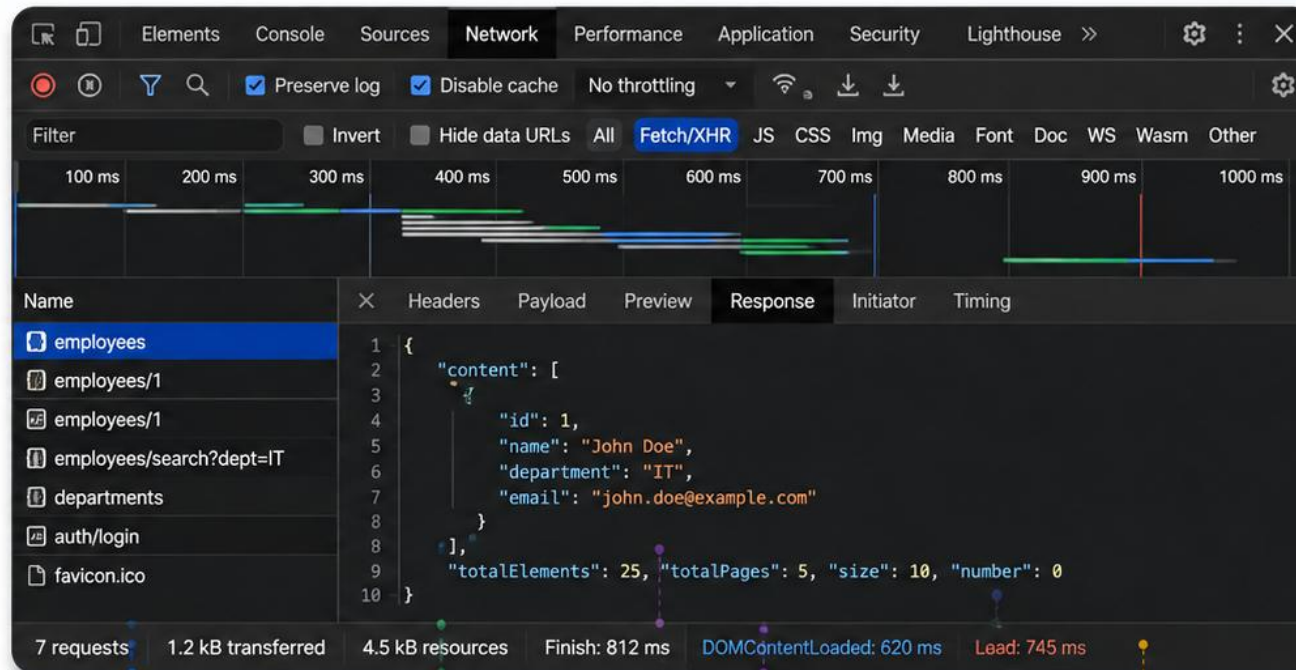


Monitor Performance

Analyze page load time, rendering performance, and identify bottlenecks.



Key Takeaway: Browser Developer Tools are essential for debugging, validating API integration, optimizing performance, and ensuring a smooth user experience.



Request List

See all network requests made by your application.

Details Tabs

Inspect headers, payload, cookies, and more.

Response Preview

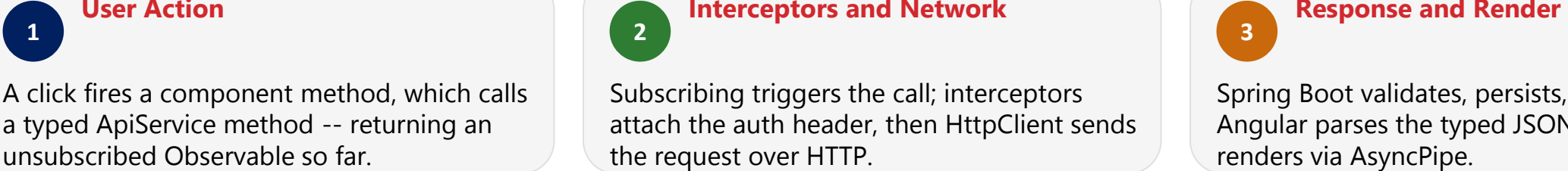
View and validate JSON responses easily.

Timing & Performance

Analyze request timing and identify slow responses.

END-TO-END DATA FLOW

One user click, traced through every layer this module covered, start to finish.



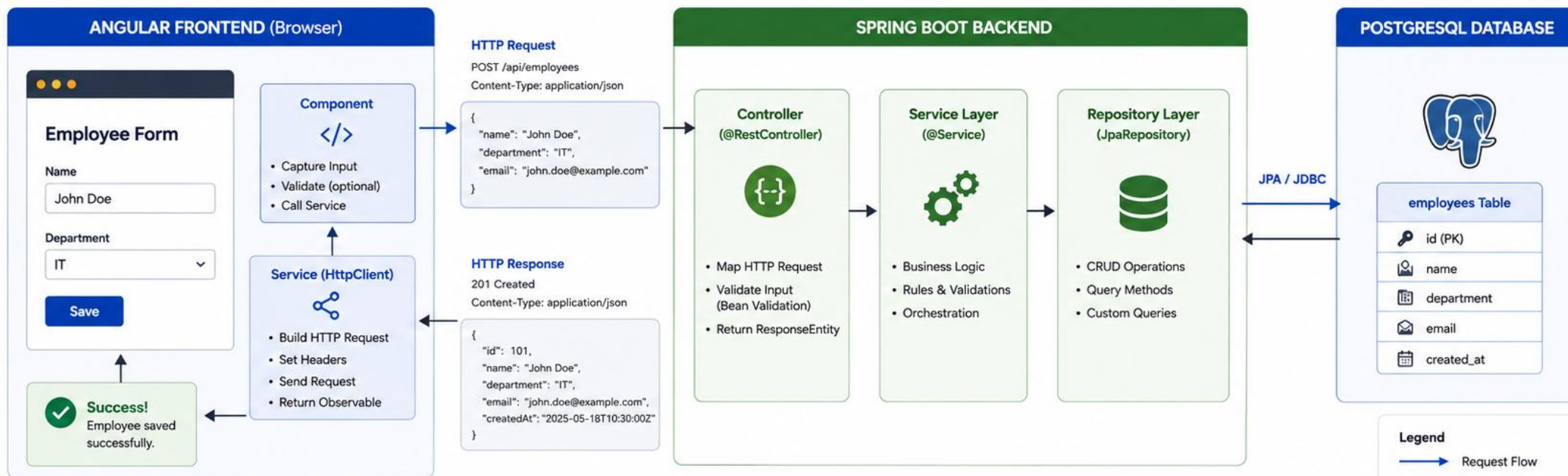
KEY TAKEAWAY Every topic in this module is one link in this exact chain -- if any single link is wrong, the whole chain fails, usually with a confusing symptom far from the actual cause.



MODULE 35 Frontend to API Integration

End-to-End Data Flow

From user action in the browser to data persistence in the database and back.



Key Takeaway:

Data flows from the user interface through the frontend layers, across the network to the backend, into the database, and back as a response to update the UI.



Secure
Communication



Validated
Transactions



Consistent
Data Flow



Reliable
Integration

ENTERPRISE INTEGRATION BEST PRACTICES

A short checklist that separates a demo-quality integration from a production-ready one.

PRACTICE	WHY IT MATTERS
Type every request and response	Catches DTO drift at compile time on the frontend; documents the contract.
Centralize auth and error handling	One interceptor, not fifty copy-pasted catchError blocks.
Model every UI state explicitly	Loading, empty, error, and success are all required, not just the happy path.
Never hardcode environment values	environment.apiUrl, not a literal URL, keeps one build deployable everywhere.
Fix CORS on the server, not the client	There is no client-side workaround for a missing Access-Control-Allow-Origin header.

KEY TAKEAWAY Every row in this checklist is a topic this module covered in depth -- this is the condensed version to carry into every future Angular feature you build.

TRY IT YOURSELF — EXERCISE 6: PLAN EVIDENCE AND SELF-CHECK

Pre-lab gate: plan the network evidence Lab 35 grades, then confirm the five earlier notes files are genuinely complete.

STEPS (notes/lab35-readiness.md)

Step	What To Do
1 — Network Evidence	Name the DevTools screenshot and what must be visible in it.
2 — End-to-End Trace	Write the hops from a click to a PostgreSQL row and back.
3 — Debug Order	Given a blank list, name the order in which you check things.
4 — Pass Mark	Mark Pass only if all five earlier notes files are complete, not stubbed.

Evidence and trace

```
screenshot: Network tab showing GET /api/customers 200
  with request header X-Correlation-Id: lab-request-001
click -> service -> interceptor -> Boot -> service -> repo -> PostgreSQL
```

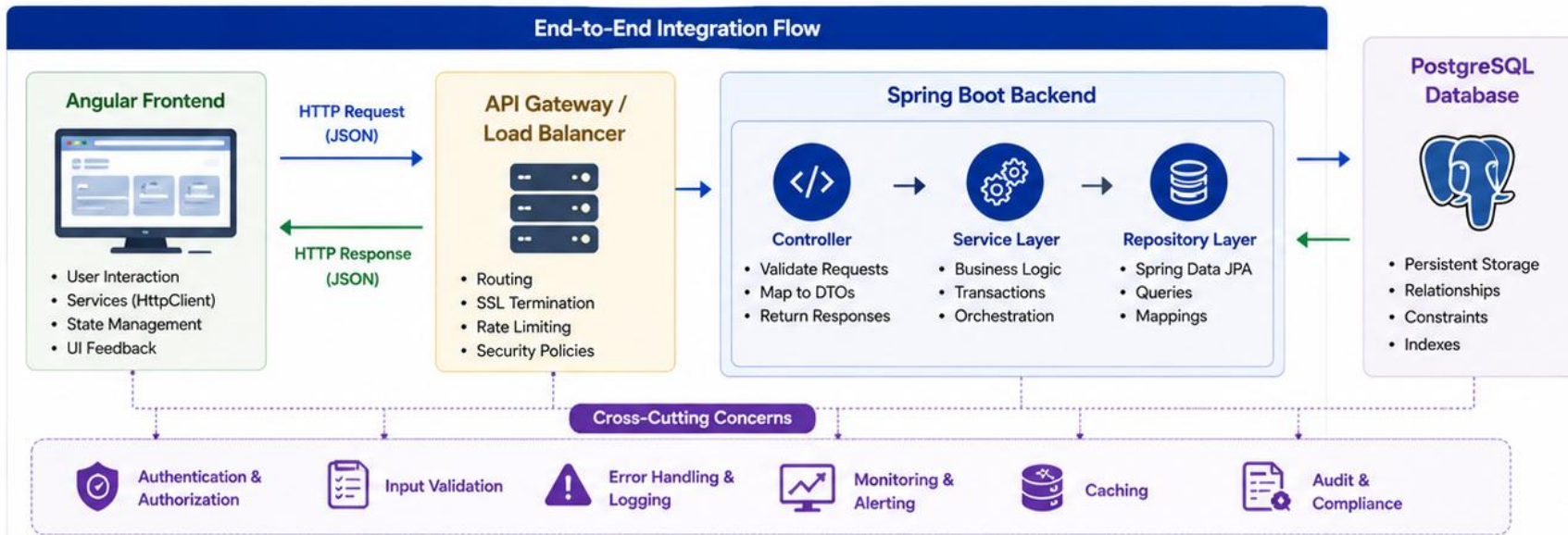
TRY IT NOW — Complete Exercise 6 before Lab 35: exercises/exercise-06-lab35-readiness.md (workspace: examples/module-35-exercises).



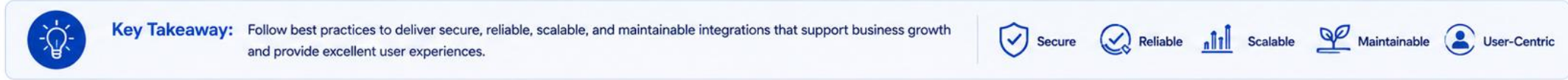
MODULE 35 Frontend to API Integration

Enterprise Integration Best Practices

Build secure, reliable, scalable, and maintainable integrations across your stack.



BEST PRACTICES CHECKLIST	
	API & Contract ✓ Use meaningful endpoints and HTTP methods ✓ Version your APIs ✓ Standardize error response format
	Security ✓ Use HTTPS everywhere ✓ Store tokens securely ✓ Validate and sanitize all inputs
	Data Management ✓ Use DTOs, not entities ✓ Handle pagination, sorting, filtering ✓ Avoid N+1 queries
	Resilience ✓ Implement retries and timeouts ✓ Handle failures gracefully ✓ Use circuit breakers if needed
	Observability ✓ Centralized logging ✓ Metrics and monitoring ✓ Track and alert on failures
	DevOps & Automation ✓ Automate builds, tests, and deployments ✓ Use CI/CD with quality gates ✓ Manage secrets securely



LAB OVERVIEW — FRONTEND—API INTEGRATION — ANGULAR → SPRING BOOT REST → POSTGRESQL

Call local Spring Boot REST from Angular with typed HttpClient services /

BUSINESS SCENARIO

- Browser speaks JSON REST only. `GET /api/customers` returns Amina and Ravi. Angular never embeds SQL. Correlation header `X-Correlation-Id: lab-request-001` on writes. Database of record is PostgreSQL — not Oracle, not SOAP.

LEARNING OBJECTIVES

- Register and use `HttpClient` Implement a typed Customer API service Align TypeScript interfaces with REST DTOs Display async results with `AsyncPipe` or Signals Configure environments and fix CORS Sketch Angular → Boot → PostgreSQL

WHAT YOU BUILD

- `examples/lab35-crm-ui/` (+ optional `lab35-crm-api/`) | List/get/create for CUS-1001 / CUS-1002 via Angular → Boot JSON (PostgreSQL or mock) |

KEY TAKEAWAY Connect the Angular CRM UI to Spring Boot REST: typed HttpClient, env base URLs, CORS for `:4200`→`:8080`, and CRUD. Behind Boot, persistence is **PostgreSQL** in the target architecture (mock repo OK if DB labs are still ahead).



MODULE 35

Frontend to API Integration

Lab Overview – Frontend and API Integration

Build a full-stack feature by connecting an Angular frontend to a Spring Boot REST API.



LAB OBJECTIVE

Integrate an Angular application with Spring Boot REST APIs to perform end-to-end CRUD operations with PostgreSQL.



TECHNOLOGY STACK



Angular 17+



TypeScript



Angular HttpClient



Spring Boot 3.x



PostgreSQL

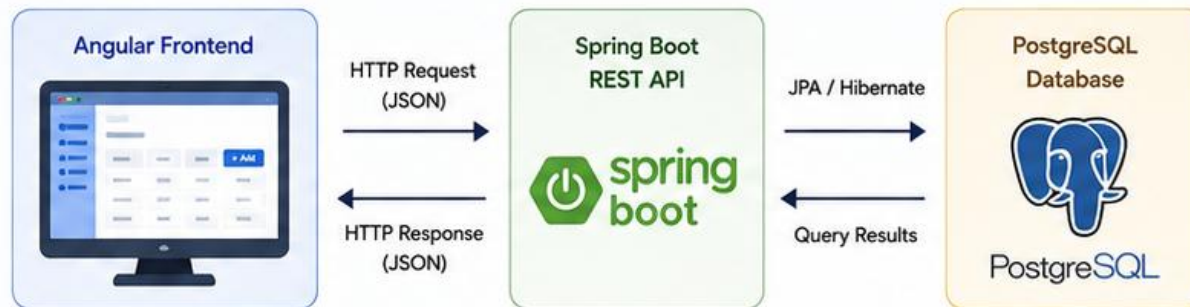


RESTful APIs



JWT Authentication

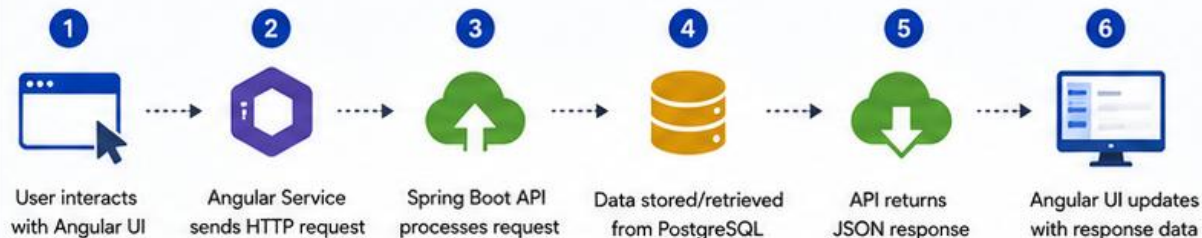
ARCHITECTURE OVERVIEW



WHAT YOU WILL BUILD

- ✓ Employee Management Feature
- ✓ Angular Service Layer
- ✓ GET, POST, PUT, PATCH, DELETE Operations
- ✓ Form Management & Validation
- ✓ Error Handling & Loading States
- ✓ JWT Token Integration
- ✓ Pagination, Filtering & Sorting
- ✓ End-to-End Data Flow

END-TO-END FLOW



KEY CONCEPTS PRACTICED

- ✓ Angular HttpClient & Observables
- ✓ REST API Consumption
- ✓ DTOs & TypeScript Interfaces
- ✓ Error & Loading State Handling
- ✓ JWT Authentication
- ✓ Cross-Origin Requests (CORS)
- ✓ Frontend-Backend Integration Patterns



LAB OUTCOMES

- A fully functional Angular frontend integrated with Spring Boot REST APIs.
- Perform complete CRUD operations with PostgreSQL persistence.
- Implement enterprise practices: error handling, JWT, pagination, filtering, and sorting.



Real-World Integration



Secure Communication



Robust Error & State Handling



Production-Ready Front-End



By the end of this lab, you will have built a complete frontend-to-backend integration flow and be ready to use it in your capstone project.

LAB TASKS AND DELIVERABLES

Frontend-API Integration — Angular → Spring Boot REST → PostgreSQL — implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Open starter/README.md.
2	Scaffold lab35-crm-ui; set env base URL to http://localhost:8080.
3	Implement CustomerApiService GET list/detail; wire list UI.
4	Run Boot + ng serve; evidence under notes/screenshots/lab-35/.
5	Mark timed-path Pass; CRUD extras as homework.
6	lab35-crm-ui with HttpClient CustomerApiService
7	TS interfaces aligned to REST customer JSON
8	List shows Amina (CUS-1001) and Ravi (CUS-1002) from API
9	Environment base URL configured
10	CORS working (or documented Boot fix)
11	Network screenshot (+ X-Correlation-Id: lab-request-001 if required)
12	docs/integration-notes.md (Angular → Boot → PostgreSQL)

EXPECTED DELIVERABLES

- lab35-crm-ui with HttpClient CustomerApiService TS interfaces aligned to REST customer JSON List shows Amina (CUS-1001) and Ravi (CUS-1002) from API Environment base URL configured CORS working (or documented Boot fix) Network screenshot (+ X-Correlation-Id: lab-request-001 if required) docs/integration-notes.md (Angular → Boot → PostgreSQL) No secrets / node_modules/ / target/ committed

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs/Evidence 10

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor Lab 35; complete pre-lab exercises 1→6 first, then the graded lab.



MODULE 35

Frontend to API Integration

Lab Tasks and Deliverables

Complete these tasks to build a fully integrated Angular + Spring Boot + PostgreSQL feature.

LAB TASKS

- Backend API Verification**
 - Verify all Employee Management REST endpoints using Postman.
 - Ensure proper status codes, validation, and error responses.
- Angular Service Integration**
 - Create Angular service using HttpClient.
 - Implement GET, POST, PUT, PATCH, DELETE methods.
- Employee List View**
 - Display employees in a table with pagination, sorting, and filtering.
 - Show loading, empty, and error states.
- Add Employee Form**
 - Build a reactive form for adding new employees.
 - Validate inputs and submit data to the API.
- Edit and Update Employee**
 - Load employee details and allow editing.
 - Update employee using PUT/PATCH API.
- Delete Employee**
 - Add delete action with confirmation dialog.
 - Remove employee using DELETE API.
- Error Handling & Messages**
 - Display user-friendly error messages for API failures.
 - Implement global HTTP error handling.
- JWT Integration**
 - Store JWT token and attach it to API requests.
 - Handle 401/403 responses and logout flow.
- End-to-End Data Flow**
 - Verify full flow from Angular UI → API → Database and back.
 - Ensure data consistency across all operations.

END-TO-END FLOW TO VALIDATE



- ✓ Create Employee → Verify in DB → Return Created Data
- ✓ Read Employees → Display in UI
- ✓ Update Employee → Verify Changes Persisted
- ✓ Delete Employee → Verify Removal in DB
- ✓ Handle Errors → Show Meaningful Messages in UI

DELIVERABLES

- Angular application with complete CRUD UI
- Angular service layer with all API methods
- Spring Boot APIs tested and working
- PostgreSQL schema and seed data
- JWT authentication integrated
- Screenshots / short demo video
- Source code pushed to GitHub
- README with setup and run instructions



TEST SCENARIOS

- ✓ Load employee list with pagination & sorting
- ✓ Add a new employee and verify in list
- ✓ Update employee details and verify
- ✓ Delete employee and verify removal
- ✓ Filter employees by department and name
- ✓ Sort by name, department, and date of joining
- ✓ Handle invalid input and API errors
- ✓ Verify JWT-protected API endpoints



SUCCESS CRITERIA

- ✓ All CRUD operations work end-to-end
- ✓ Data is persisted in PostgreSQL
- ✓ UI is responsive and user-friendly
- ✓ Errors are handled gracefully
- ✓ Code is clean, modular, and committed



Goal: Deliver a robust integration between the Angular frontend and Spring Boot REST API with PostgreSQL persistence, implementing enterprise best practices for error handling, security, and maintainability.



Secure



Reliable



Performant



Maintainable

MODULE 35 SUMMARY

Angular now calls a typed, secured, well-behaved Spring Boot API through one consistent service pattern.

KEY CONCEPTS COVERED



Typed HttpClient

Every call is `Observable<T>` for a specific `T`, matching a TypeScript interface that mirrors the backend DTO.



Observables + AsyncPipe

Cold Observables only fire on subscribe; AsyncPipe subscribes and unsubscribes automatically in templates.



Every UI State

Loading, empty, error, and success are all modeled explicitly -- never just the happy path.



Interceptors

One auth interceptor and one error interceptor handle token injection and global failure handling app-wide.



Environment Config

`environment.apiUrl` and `fileReplacements` keep one Angular build deployable to dev, test, and production.



CORS

Cross-origin access is a server-side opt-in via Spring Boot's `CorsConfigurationSource` -- never an Angular-side fix.

MODULE FLOW RECAP

1

Type the Contract

2

Call & Subscribe

3

Handle Every State

4

Secure & Configure

100

Debug & Verify

KEY TAKEAWAY A production-ready Angular-to-Spring-Boot integration is typed end to end, models every UI state explicitly, centralizes auth and error handling in interceptors, and never hardcodes an environment value or fights CORS from the client side.



MODULE 35

Frontend to API Integration

Module Summary

You connected the frontend, backend, and database to deliver **real data and value**.

WHAT WE COVERED



REST Communication

Exchanged JSON data using HTTP methods with the Spring Boot REST API.



Angular HttpClient

Used Angular HttpClient and Observables to call APIs efficiently.



Service Layer

Built a clean service layer to encapsulate API calls and handle responses.



CRUD Operations

Implemented full CRUD workflow and connected Angular forms to the backend.



Error Handling

Handled API errors, validation messages, and global error responses.



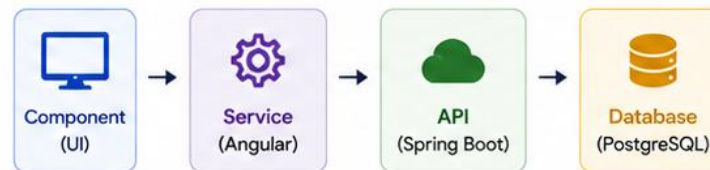
End-to-End Flow

Achieved complete data flow from Angular UI → Spring Boot API → PostgreSQL.

KEY TAKEAWAYS

- ✓ Frontend and backend communicate using RESTful APIs.
- ✓ Typed models and DTOs ensure consistency.
- ✓ Services and Observables promote clean and scalable code.
- ✓ User experience is improved with loading, empty, and error states.
- ✓ Security and configuration are managed with interceptors, CORS, and environment-based URLs.

COMPONENT → SERVICE → API FLOW



- ✓ Component triggers action (e.g., form submit, button click).
- ✓ Service sends HTTP request to the Spring Boot API.
- ✓ API processes request and interacts with the PostgreSQL database.
- ✓ Response is returned and consumed in the UI.

YOU NOW CAN

- Integrate Angular with any REST API confidently.
- Manage data, state, and errors effectively.
- Implement secure and reliable API communication.
- Deliver responsive and real-time user experiences.



In a Nutshell: You built the bridge between the frontend and backend, enabling real data interactions and laying a strong foundation for scalable, secure, and enterprise-grade full-stack applications.



Integrated



Reliable



Performant



Secure



User-Centric

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of HttpClient, Observables, typed responses, and CORS.

1. What actually triggers an HttpClient GET request to be sent to the server?

- A. Calling the service method that builds the Observable
- B. Calling .subscribe() on the returned Observable**
- C. Declaring the method's return type as Observable<T>
- D. Importing HttpClientModule

3. A CORS error appears in the browser console for a request to the Spring Boot API. What is the correct fix?

- A. Add a try/catch around the HttpClient call in Angular
- B. Switch the request from GET to POST
- C. Configure allowed origins on the Spring Boot side (e.g. a CorsConfigurationSource bean)**
- D. Disable the browser's security settings

2. Why should createCustomer() (a POST) never be silently auto-retried on failure?

- A. POST requests are always slower than GET requests
- B. POST is not idempotent -- a retry can create a duplicate record**
- C. Angular's HttpClient does not support retrying POST at all
- D. Spring Boot rejects any repeated POST automatically

4. Why does the Angular team recommend `customers\$ | async` over a manual .subscribe() in a template?

- A. It performs the HTTP request faster
- B. It automatically subscribes and unsubscribes, avoiding memory leaks**
- C. It is the only syntax that works with HttpClient
- D. It disables RxJS operators for that call

KEY TAKEAWAY A request only fires on subscribe; POST is never safely auto-retried; CORS is fixed on the Spring Boot side; and AsyncPipe is preferred because it manages subscription lifecycle automatically.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on DTO alignment, interceptors, UI states, and environment configuration.

5. A backend enum `CustomerStatus` serializes to JSON as `"ACTIVE"`. What is the correct Angular representation?

- A. A TypeScript enum with matching numeric values
- B. A string literal union type, e.g. `'ACTIVE' | 'INACTIVE' | 'PROSPECT'`**
- C. A plain string with no type restriction at all
- D. A boolean flag for each possible status

7. A list screen shows nothing after a request that returned 200 OK with an empty array. What is the correct UI behavior?

- A. Keep showing the loading spinner indefinitely
- B. Show a clear empty-state message distinct from both loading and error**
- C. Show the same generic error banner used for a failed request
- D. Silently render nothing, since there is no data to show

6. Why should the JWT Authorization header be attached by an interceptor rather than inside each service method?

- A. Interceptors run faster than service method code
- B. It guarantees the header is attached consistently everywhere, with no risk of forgetting it in one place**
- C. Service methods cannot set custom headers in Angular
- D. Interceptors are required by TypeScript's type system

8. Why must an Angular app read its backend URL from `environment.apiUrl` instead of a hardcoded string?

- A. Hardcoded strings are not valid TypeScript syntax
- B. HttpClient rejects any URL that isn't a template literal
- C. It lets the same build swap URLs per environment via angular.json's fileReplacements**
- D. It makes the app run faster in the browser

KEY TAKEAWAY Enums become string literal unions on the wire; interceptors centralize the auth header for consistency; an empty successful response needs its own explicit UI state; and `environment.apiUrl` is what makes one build deployable everywhere.



MODULE 35

Frontend to API Integration

Knowledge Check

Test your understanding of frontend-backend integration concepts.



Choose the best answer for each question.

1 Which Angular service is used to make HTTP requests?

- A Router
- B **HttpClient** ✓
- C FormBuilder
- D EventEmitter

2 What is the default return type of HttpClient methods?

- A Promise
- B **Observable<T>** ✓
- C Future<T>
- D Stream<T>

3 Which HTTP method should be used to update an existing resource partially?

- A PUT
- B POST
- C **PATCH** ✓
- D DELETE

4 What header is commonly used to send JWT tokens to the server?

- A Content-Type
- B Accept
- C **Authorization** ✓
- D X-Requested-With

5 Which Angular feature automatically unsubscribes from observables when the component is destroyed?

- A HttpInterceptor
- B **AsyncPipe** ✓
- C BehaviorSubject
- D take(1)

6 If the API returns HTTP status 201, what does it indicate?

- A Resource retrieved successfully
- B **Resource created successfully** ✓
- C Request is invalid
- D Internal server error

7 Which of the following is used to share configuration values like API base URLs across environments?

- A HttpHeaders
- B **environment.ts** ✓
- C AppComponent
- D routes.ts

8 What is the purpose of an HTTP Interceptor?

- A To render UI templates
- B **To modify requests or responses globally** ✓
- C To handle routing
- D To validate forms

9 Which file configures CORS on the Spring Boot backend?

- A application.properties
- B **CorsFilter.java / WebConfig.java** ✓
- C application.yml
- D index.html

10 Which of the following best describes an end-to-end data flow?

- A UI → Service → Database
- B **Component → Service → API → Database → UI** ✓
- C API → UI → Service
- D Database → Component → API



Think Beyond

Explain how error handling, loading states, and interceptors improve the reliability and user experience of an Angular application.



Scoring Guide

- 8 – 10 Correct: Excellent! You have a strong understanding.
- 5 – 7 Correct: Good job! Review the topics and try again.
- 0 – 4 Correct: Keep learning! Revisit the module sections.



Key Takeaway

A well-designed integration ensures secure, reliable, and efficient communication between the frontend, backend, and database.



Great work! You're building the skills to deliver powerful full-stack applications.



Secure



Reliable



Performant



Integrated



User-Centric

Frontend to API Integration

You can now build a typed, observable-based Angular service layer that calls a secured Spring Boot API and handles every outcome gracefully.

